

Тема 5.3 Работа с данными

Отправка форм

Одной из форм отправки данных на сервер представляет отправка с помощью форм html, обычно в запросе типа POST. В Django в целом можно использовать два подхода для работы с формами. Во-первых, можно работать со стандартными формами html. Во-вторых, Django также предоставляет специальный функционал для работы с формами, который, возможно, в каких-то моментах упрощает работу с данными. В данной главе рассмотрим оба подхода. А в данной статье посмотрим, как получать данные обычных форм html.

Обычно данные форм передаются на сервер в запросе типа POST. Для получения подобных данных в классе `HttpRequest` определено свойство `POST`. Например, пусть у нас есть следующий шаблон `index.html`:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>METANIT.COM</title>
</head>
<body>
  <h2>User form</h2>
  <form method="post" action="postuser/">
    {% csrf_token %}
    <p>Name:<br> <input name="name" /></p>
    <p>Age:<br> <input name="age" type="number" /></p>
    <input type="submit" value="Send" />
  </form>
</body>
</html>
```

Здесь определена форма условно для ввода данных пользователя, которая в запросе типа POST (атрибут `method="post"`) отправляет данные по адресу `"postuser/"` (атрибут `action="postuser/"`).

На форме определены два поля ввода. Первое поле предназначено для ввода имени пользователя. Второе поле - для ввода возраста пользователя.

Также внутри формы используется тег `{% csrf_token %}`. Он позволяет защитить приложение от CSRF-атак, добавляя в форму в виде скрытого поля csrf-токен. Кроме того, Django по умолчанию требует наличия данного токена в получаемых данных в запросе POST.

Для отправки формы и получения ее данных определим в файле `views.py` следующие функции:

```
from django.shortcuts import render
from django.http import HttpResponse
def index(request):
    return render(request, "index.html")
def postuser(request):
    # получаем из данных запроса POST отправленные через форму данные
```

```

name = request.POST.get("name", "Undefined")
age = request.POST.get("age", 1)
return HttpResponse(f"<h2>Name: {name} Age: {age}</h2>")

```

В представлении `index` возвращается шаблон, который содержит форму ввода.

В представлении `postuser` получаем через словарь `request.POST` отправленные из формы данные. В этом словаре по ключу можно получить значение элемента. При этом в качестве ключей выступает названия полей форм (значения атрибутов `name` элементов формы):

```
<input name="age" type="number" />
```

Так, в данном случае название поля (значение атрибута `name`) равно `"age"`. Соответственно в `request.POST` по этому имени мы можем получить его значение:

```
age = request.POST.get("age", 1)
```

Если по каким-то причинам данные с ключом `"age"` в запросе отсутствуют, то возвращается значение по умолчанию - 1.

Далее в файле `urls.py` свяжем эти функции с маршрутами:

```

from django.urls import path
from hello import views
urlpatterns = [
    path("", views.index),
    path("postuser/", views.postuser),
]

```

И после получения данных формы они отправляются обратно клиенту (рисунок 1):

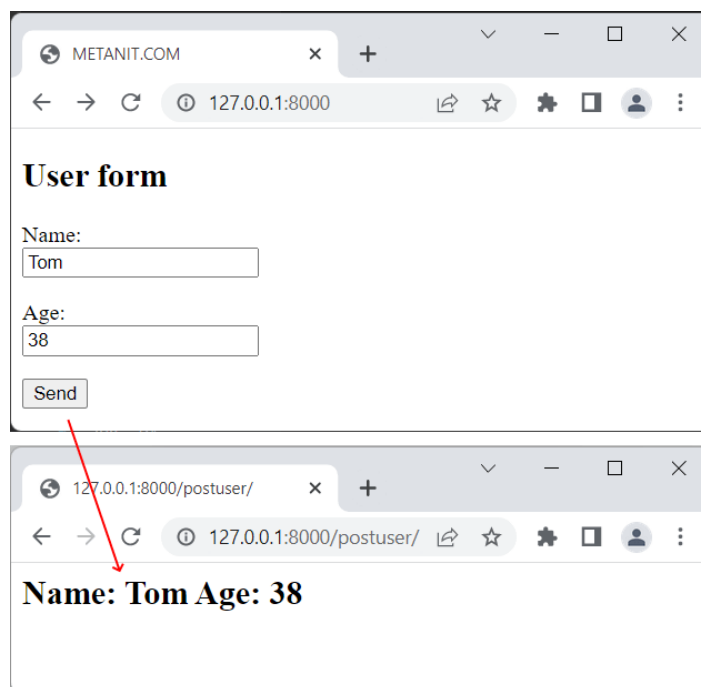


Рисунок 1 - Отправка форм в веб-приложении на Django и Python

Получение массивов

Усложним задачу и добавим в форму на странице index.html несколько полей, которые будут представлять массив:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>METANIT.COM</title>
</head>
<body>
  <h2>User form</h2>
  <form method="post" action="postuser/">
    {% csrf_token %}
    <p>Name:<br /> <input name="name" /></p>
    <p>Age:<br /> <input name="age" type="number" /></p>
    <p>
      Languages:<br />
      <input name="languages" /><br />
      <input name="languages" /><br />
      <input name="languages" /><br />
    </p>
    <input type="submit" value="Send" />
  </form>
</body>
</html>
```

Здесь практически та же форма, только добавлено три поля для ввода языка программирования. Причем каждое из этих полей имеет одно и то же имя - "languages". Благодаря этому при отправке формы в данных запроса будет сформирован список languages из данных, введенных в эти поля.

В файле views.py изменим функцию postuser для получения массива languages:

```
from django.shortcuts import render
from django.http import HttpResponse
def index(request):
    return render(request, "index.html")
def postuser(request):
    # получаем из строки запроса имя пользователя
    name = request.POST.get("name", "Undefined")
    age = request.POST.get("age", 1)
    langs = request.POST.getlist("languages", ["python"])
    return HttpResponse(f"""
        <div>Name: {name} Age: {age}</div>
        <div>Languages: {langs}</div>
    """)
```

Ключевой компонент при получении списка данных из запроса представляет метод getlist(), который работает так же, как и get(), только

возвращает список. Если в запросе не окажется данных с ключом languages, то возвращаем список ["python"]

Получив список, мы можем что-то сделать с его элементами, перебрать, обратиться к элементам по индексу и т.д. Но в данном случае просто передаем весь список в формируемый ответ (рисунок 2).

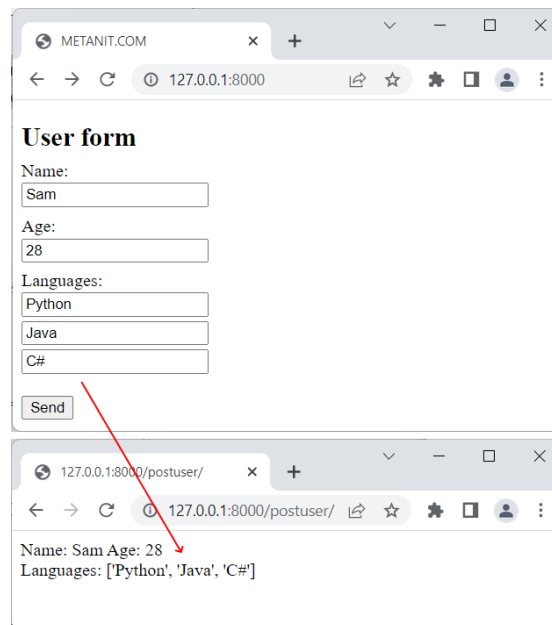


Рисунок 2 - Отправка массивов и списков в Django в запросе Post

Подобным образом можно передавать значения массива полей других типов, либо полей, которые представляют набор элементов, например, элемента select, который поддерживает множественный выбор (рисунок 3):

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>METANIT.COM</title>
</head>
<body>
  <h2>User form</h2>
  <form method="post" action="postuser/">
    {% csrf_token %}
    <p>Name: <br />
      <input name="name" />
    </p>
    <p>Age: <br />
      <input name="age" type="number" />
    </p>
    <p>
      Languages:<br />
```

```

<select multiple name="languages">
  <option>Python</option>
  <option>JavaScript</option>
  <option>C++</option>
  <option>Java</option>
</select>
</p>
<input type="submit" value="Send" />
</form>
</body>
</html>

```

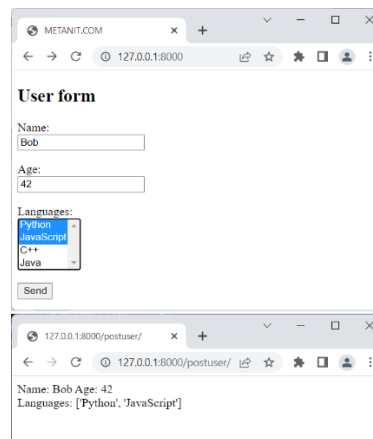


Рисунок 3 - Получение списков в Django в запросе Post

Создание форм и обработка запросов в Django

Django предоставляет ряд инструментов и библиотек, которые помогут вам создавать формы, принимающие ввод от посетителей сайта, а затем обрабатывающие вводимые данные и отвечающие на них.

В HTML, форма — это набор элементов внутри, **<form>...</form>**, которые позволяют посетителю делать такие вещи, как ввод текста, выбирать параметры, манипулировать объектами или элементами управления и т. Д., а затем отправлять эту информацию обратно на сервер.

Некоторые из этих элементов интерфейса формы - ввод текста или флажки - встроены в сам HTML. Другие намного сложнее; интерфейс, который открывает окно выбора даты или позволяет перемещать ползунок или управлять элементами управления, обычно использует JavaScript и CSS, а также **<input>** элементы формы HTML для достижения этих эффектов.

Помимо своих **<input>** элементов, форма должна определять две вещи:

- *где*: URL-адрес, по которому должны быть возвращены данные, соответствующие вводу пользователя;

- *как*: HTTP-метод, который должен возвращать данные

GET и POST

GET и **POST** являются единственными методами HTTP, которые можно

использовать при работе с формами.

Форма входа в Django возвращается с использованием **POST** метода, в котором браузер объединяет данные формы, кодирует их для передачи, отправляет на сервер, а затем получает ответ.

GET, напротив, объединяет представленные данные в строку и использует ее для составления URL-адреса. URL-адрес содержит адрес, по которому должны быть отправлены данные, а также ключи и значения данных. Вы можете увидеть это в действии, если выполните поиск в документации Django, и в результате будет получен URL-адрес формы

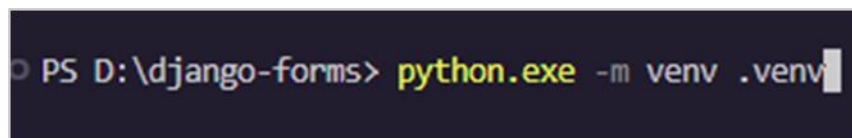
GET и **POST** обычно используются для разных целей.

Любой запрос, который можно использовать для изменения состояния системы - например, запрос, который вносит изменения в базу данных - должен использовать **POST**. **GET** следует использовать только для запросов, не влияющих на состояние системы.

GET также не подходит для формы пароля, потому что пароль будет отображаться в URL-адресе и, следовательно, также в истории браузера и журналах сервера, все в виде обычного текста. Он также не подходит для больших объемов данных или для двоичных данных, таких как изображение. Веб-приложение, использующее **GET** запросы для форм администратора, представляет собой угрозу безопасности: злоумышленник может легко имитировать запрос формы для получения доступа к конфиденциальным частям системы. **POST**, в сочетании с другими средствами защиты, такими как защита Django от [CSRF](#), обеспечивает больший контроль над доступом.

С другой стороны, **GET** он подходит для таких вещей, как форма веб-поиска, потому что URL-адреса, которые представляют **GET** запрос, можно легко добавить в закладки, поделиться или повторно отправить.

Для начала создадим Django-проекта и полностью настроим отображение для страницы index.html (рисунок 4-5).



```
PS D:\django-forms> python.exe -m venv .venv
```

Рисунок 4 - Установка виртуального окружения



```
PS D:\django-forms> python.exe -m venv .venv
PS D:\django-forms> .\.venv\Scripts\activate
(.venv) PS D:\django-forms> pip install django
Collecting django
  Using cached Django-5.0.2-py3-none-any.whl.metadata (4.1 kB)
Collecting asgiref<4,>=3.7.0 (from django)
```

Рисунок 5 - Установка django

Далее, создадим новый проект `django_forms_learning` (рисунок 6):

```
(.venv) PS D:\django-forms> django-admin startproject django_forms_learning
(.venv) PS D:\django-forms> 
```

Рисунок 6 - Создание нового проекта `django_forms_learning`

Перейдем в неё (рисунок 7):

```
(.venv) PS D:\django-forms> dir

Каталог: D:\django-forms

Mode                LastWriteTime         Length Name
----                -
d-----          27.02.2024   18:57             .venv
d-----          27.02.2024   18:59      django_forms_learning

(.venv) PS D:\django-forms> cd .\django_forms_learning\
```

Рисунок 7 - Переход в директорию `django_forms_learning`

Создадим новое приложение. Назовем его `index` (рисунок 8):

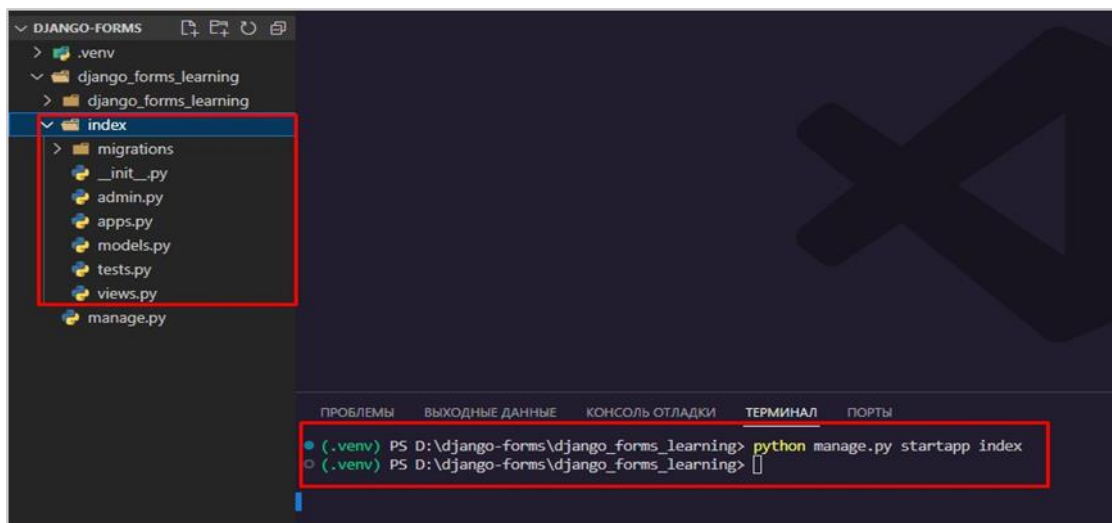


Рисунок 8 - Создание нового django-app (django-приложения)

Зарегистрируем приложение в файле `settings.py` (в `INSTALLED_APPS`) (рисунок 9):

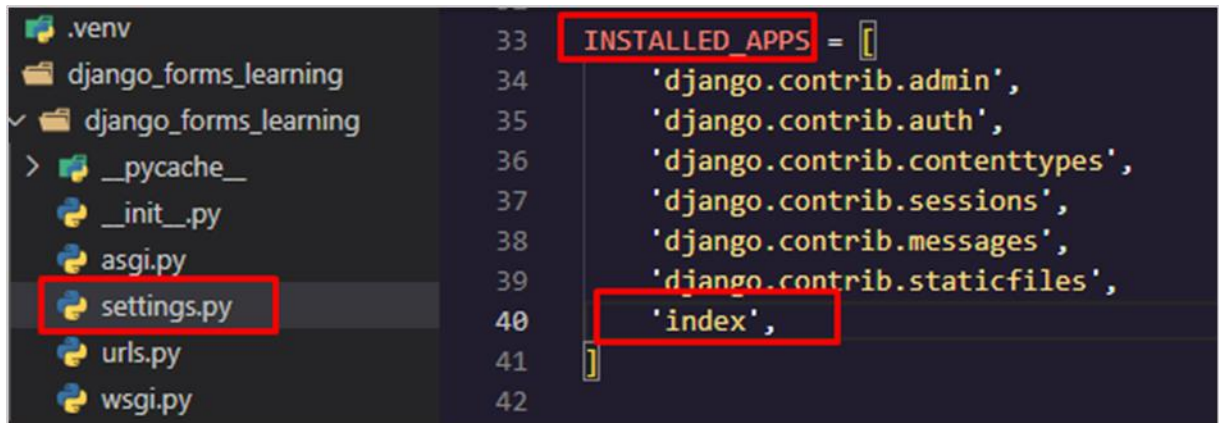


Рисунок 9 - Добавление в INSTALLED_APPS приложения index

Создадим константу, для хранения шаблонов (templates) (рисунок 10):



Рисунок 10 - Добавление директории для шаблонов (templates)

Мы использовали os.path, и для того, чтобы все работало, необходимо импортировать модуль os. Для этого пропишем «import os» (рисунок 11):

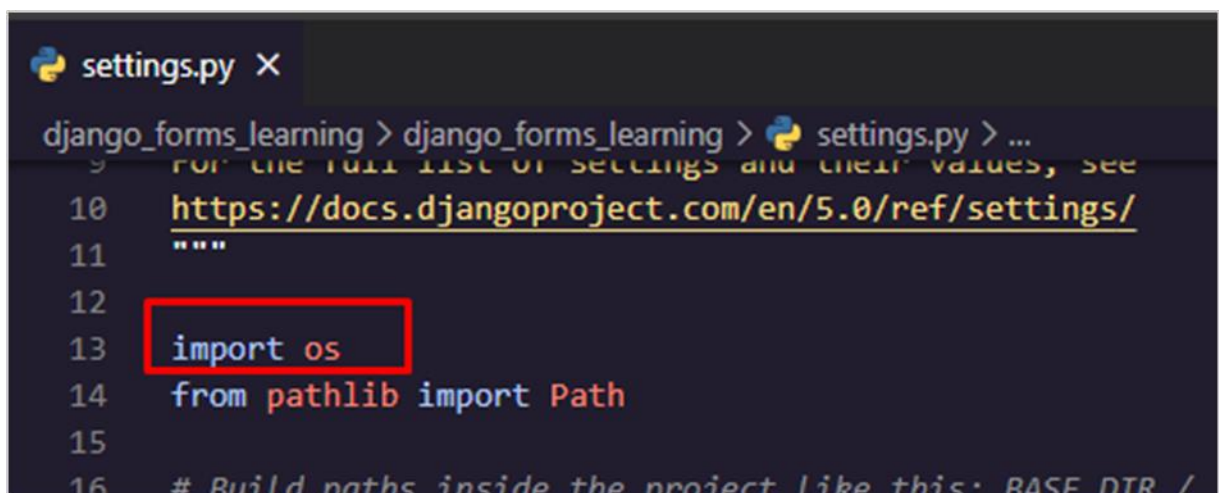


Рисунок 11 - Импортирование модуля os

Ваша иерархия проекта должна выглядеть следующим образом (рисунок 12):

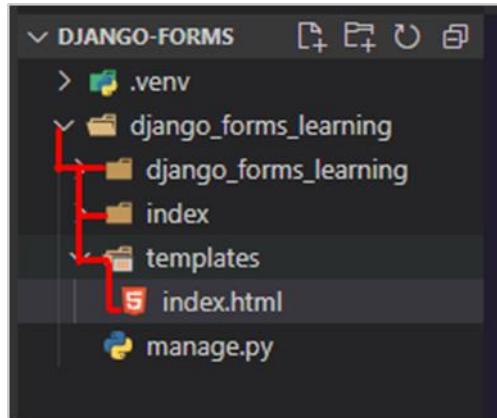


Рисунок 12 - Правильная иерархия проекта

Для красоты добавим стили Bootstrap через ссылку (CDN подключение) (рисунок 13):

```
index.html X
django_forms_learning > templates > index.html > html
1 <!doctype html>
2 <html lang="en">
3 <head>
4 <meta charset="utf-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1">
6 <title>Forms Learning</title>
7 <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet"
  integrity="sha384-QmTKZyjpPEjISv5MaRU9OFerPoke6YctnYmDr5pNly12bRjXh0JMHjYehN+ALEwIH" crossorigin="anonymous">
8 </head>
9 <body>
10 <h1>Обучение формам!</h1>
11
12 <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"
  integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9F0VZLE5aAA55NDz0xhy9GkcIdslK1eN7N6jIeH2" crossorigin="anonymous"></script>
13 </body>
14 </html>
```

Рисунок 13 - HTML-код с подключением Bootstrap через CDN

Далее, добавим вью-функцию для отображения страницы index.html (рисунок 14):

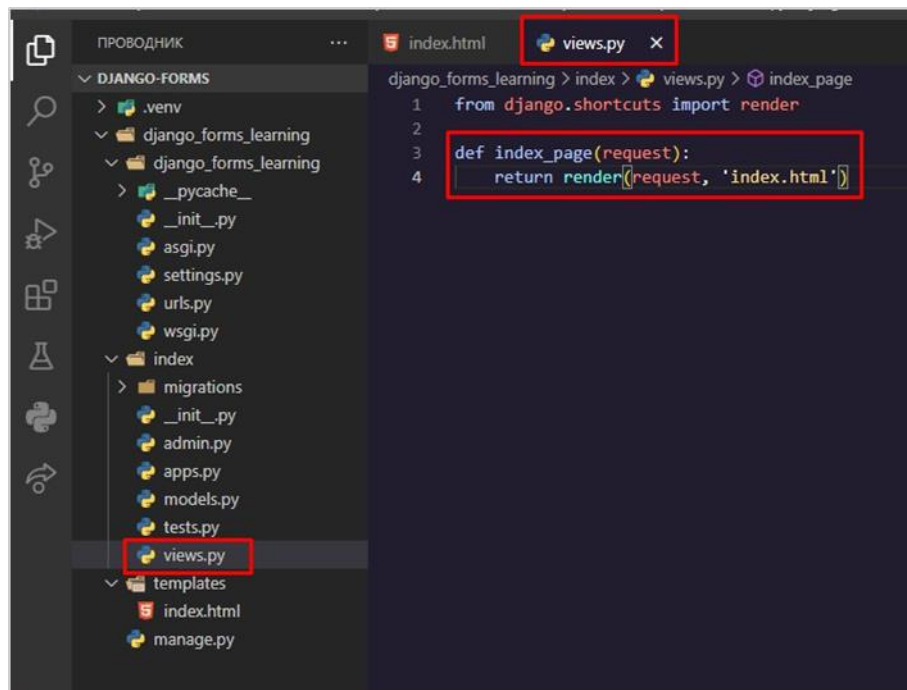


Рисунок 14 -Создание व्यю-функции для отображения главной страницы

Чтобы व्यю-функция работала, ей нужен задать маршрут. Зададим его на главную страницу, указав `path("", index_page)` (рисунок 15):

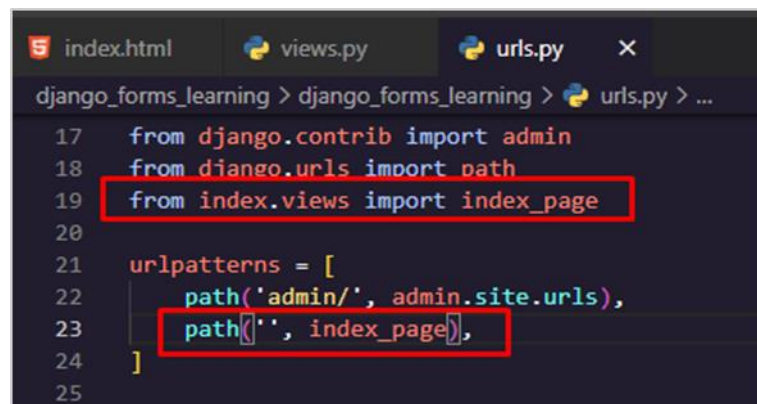


Рисунок 15 - Добавление маршрута в файле urls.py

В завершение, запустим локальный сервер (рисунок 16):

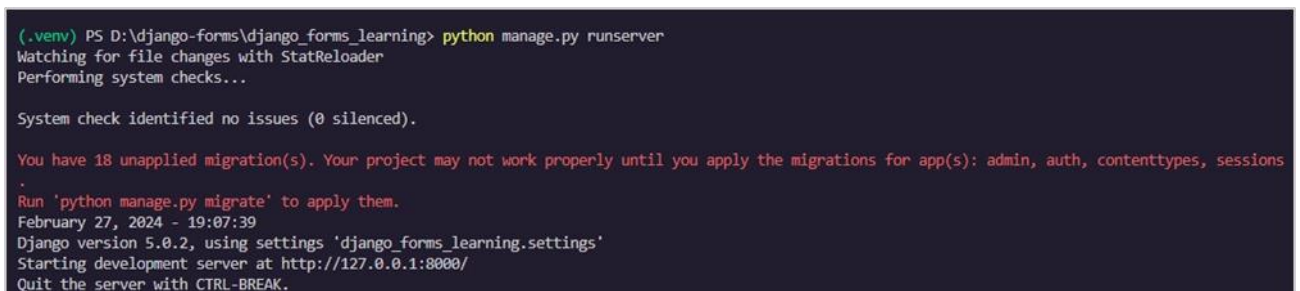


Рисунок 16 - Запуск локального сервера

Далее, после всех проделанных действий создадим форму. Для этого создадим в приложении index файл под названием forms.py и пропишем следующую логику (рисунок 17):

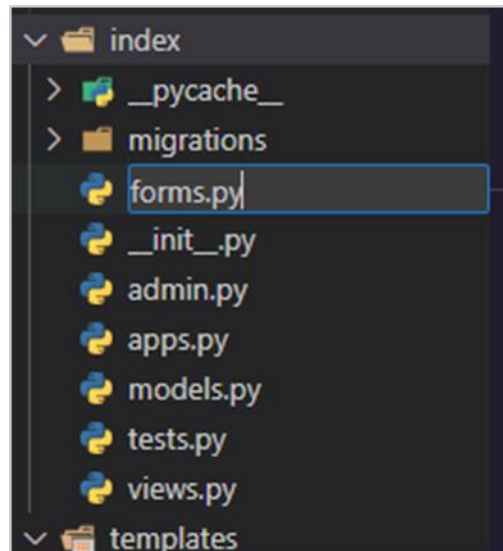


Рисунок 17 - Создание файла для форм (forms.py)

Изменим файл forms.py и добавим поля для ввода ФИО и возраста. Поля здесь вводятся по аналогии с моделями, только указывается не models, а forms (рисунок 18):

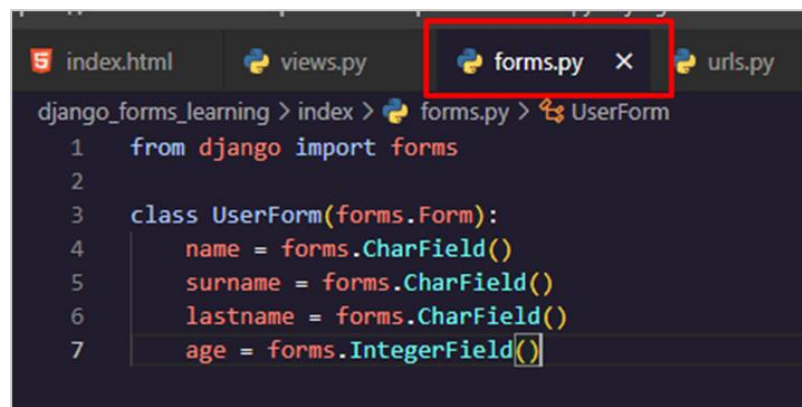


Рисунок 18 - Добавление полей для формы

Передадим форму в контекст, чтобы форма могла отображаться на странице. Для этого изменим вью-функцию, добавим импорт формы из приложения index, а также создадим переменную для хранения объекта формы UserForm (рисунок 19).

```

1 from django.shortcuts import render
2 from index.forms import UserForm
3
4 def index_page(request):
5     userform = UserForm()
6     return render(request, 'index.html', {'form': userform})
7

```

Рисунок 19 - Изменение вью-функции (добавление формы в контекст)

Вспоминая Django Template Language (DTL), отобразим форму на странице (рисунок 20):

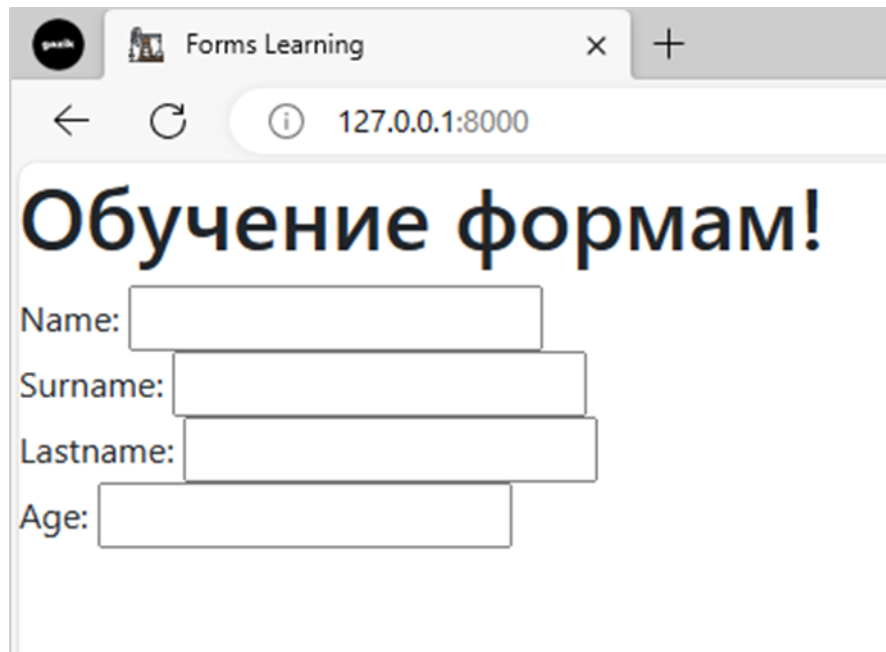
```

1 <!doctype html>
2 <html lang="en">
3 <head>
4 <meta charset="utf-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1">
6 <title>Forms Learning</title>
7 <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet"
8 integrity="sha384-QWTKZyjpPEjISv5MaRU9OFerRpk6YctnYmDr5pNlyT2bRjXh0JMhJY6Hh+ALEwIH" crossorigin="anonymous">
9 </head>
10 <body>
11 <h1>Обучение формам!</h1>
12
13 {{ form }}
14
15
16
17
18
19
20 <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"
21 integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9F0VZLESaAAS5SNDzOxhy9GkcIdslK1eN7N6jIeHz" crossorigin="anonymous"></script>
22 </body>
23 </html>

```

Рисунок 20 - Отображение формы на страницы, используя DTL

В конечном итоге, мы получим следующее (рисунок 21):



Forms Learning

127.0.0.1:8000

Обучение формам!

Name:

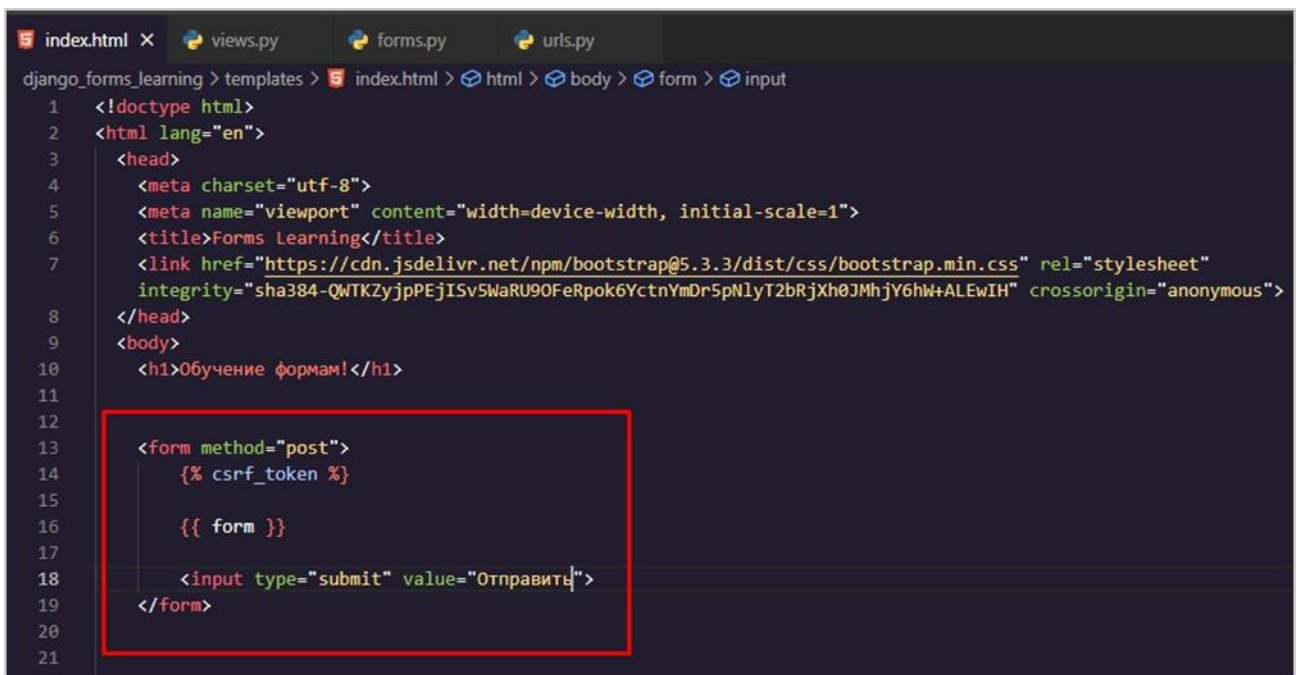
Surname:

Lastname:

Age:

Рисунок 21- Форма, отображенная на странице

Чтобы отправлять данные на сервер с помощью, нужно установить атрибут `method` в форме на «post». По умолчанию используется метод GET. Также, необходимо добавить кнопку, либо поле `input` с атрибутом «submit» (рисунок 22).



```

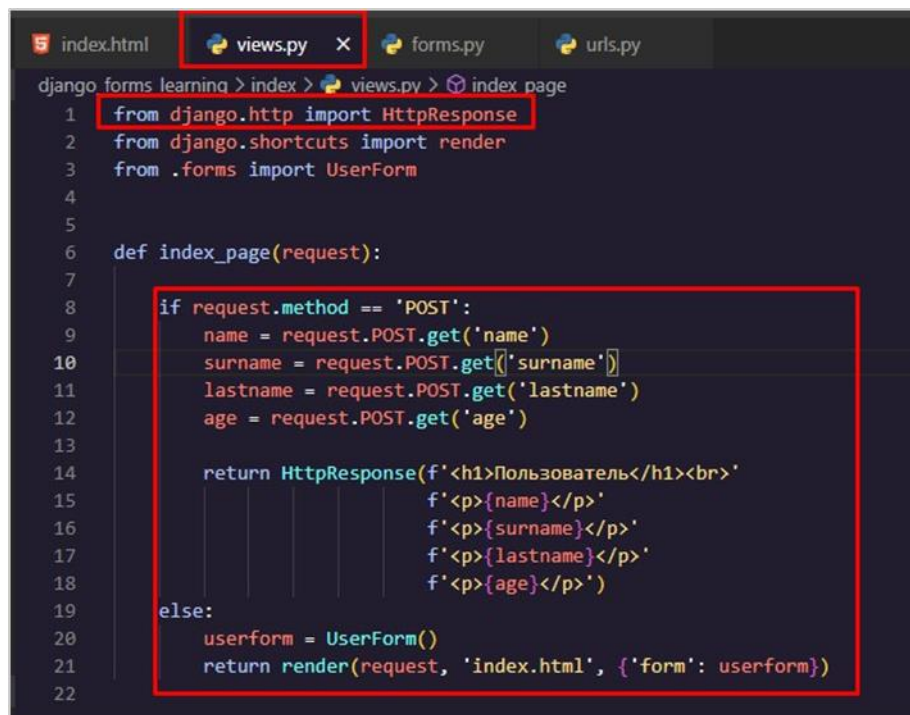
1  <!doctype html>
2  <html lang="en">
3    <head>
4      <meta charset="utf-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1">
6      <title>Forms Learning</title>
7      <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet"
8        integrity="sha384-QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh0JMhY6hW+ALEwIH" crossorigin="anonymous">
9    </head>
10   <body>
11     <h1>Обучение формам!</h1>
12
13     <form method="post">
14       {% csrf_token %}
15
16       {{ form }}
17
18       <input type="submit" value="Отправить">
19     </form>
20
21

```

Рисунок 22 - Форма с post-запросом

Будем отправлять HTTP-ответ (`HttpResponse`) при отправке формы. Добавим проверку, что если метод нашего запроса POST, то у нас будет из полей формы с названиями `name`, `surname`, `lastname` и `age` брать значения и возвращаться пользователю в виде `HttpResponse`. Для этого изменим код следующим образом

(рисунок 23):



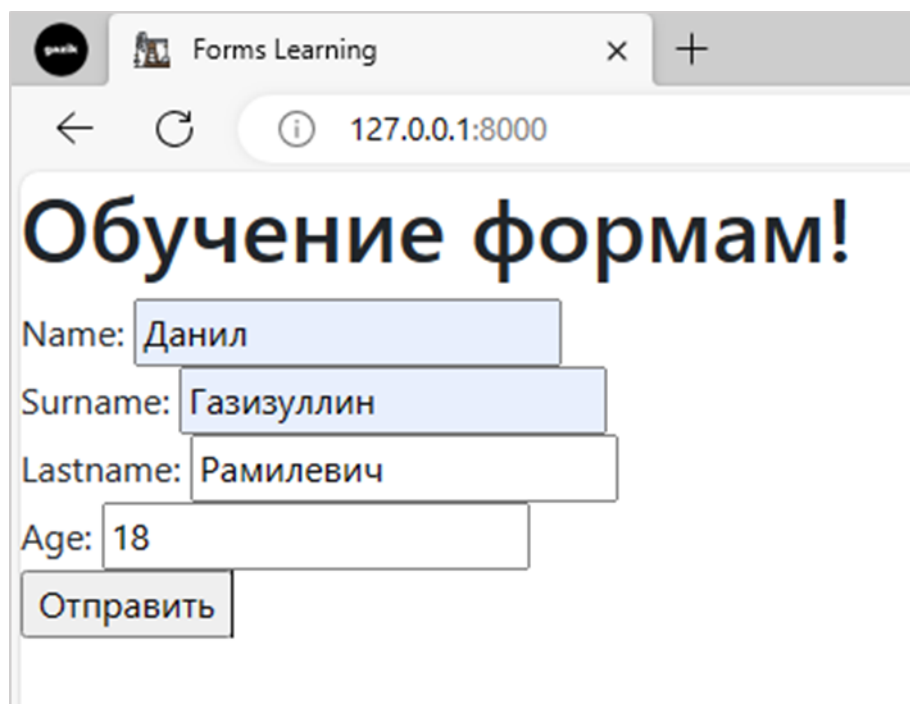
```

1  from django.http import HttpResponseRedirect
2  from django.shortcuts import render
3  from .forms import UserForm
4
5
6  def index_page(request):
7
8      if request.method == 'POST':
9          name = request.POST.get('name')
10         surname = request.POST.get('surname')
11         lastname = request.POST.get('lastname')
12         age = request.POST.get('age')
13
14         return HttpResponseRedirect(f'<h1>Пользователь</h1><br>'
15                                     f'<p>{name}</p>'
16                                     f'<p>{surname}</p>'
17                                     f'<p>{lastname}</p>'
18                                     f'<p>{age}</p>')
19     else:
20         userform = UserForm()
21         return render(request, 'index.html', {'form': userform})
22

```

Рисунок 23 - HTTP-ответ от сервера пользователю (HTTP Response)

После проделанных действий, на странице отобразится форма с кнопкой отправить. Заполним её и нажмем «Отправить» (рисунок 24).



Обучение формам!

Name:

Surname:

Lastname:

Age:

Рисунок 24- Отображение формы на странице (заполненной)

После отправки формы мы получим следующий результат (рисунок 25):

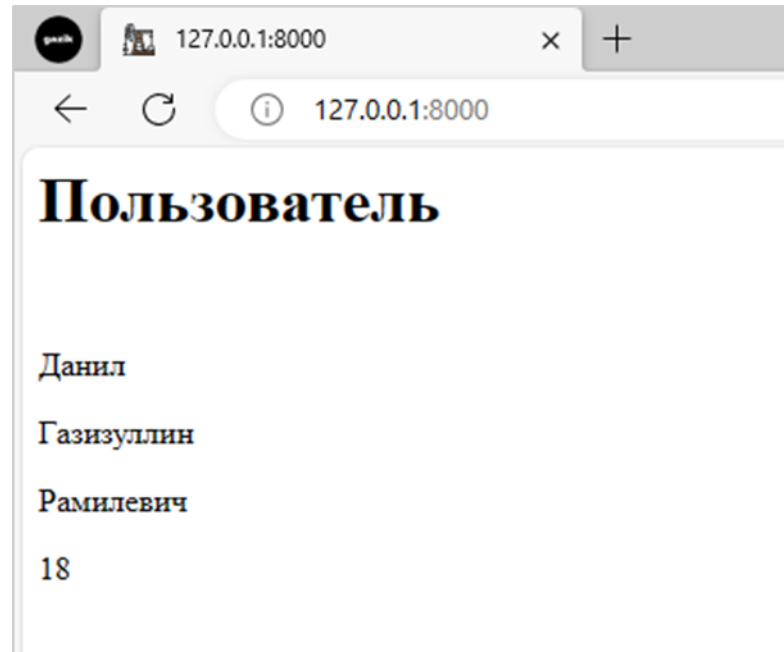


Рисунок 25 - HTTP-ответ от сервера

Пример решения задачи на языке программирования Python

Задача. В ранее созданном проекте `blog_site` и приложении `posts` создайте форму добавления публикации с полями «Заголовок» и «Текст публикации». Реализуйте обработку введенных данных во `views.py`. После отправки формы сохраните публикацию и выведите введенные данные на отдельной странице результата.

Решение задачи:

```
1 models.py
from django.db import models
class Post(models.Model):
    title = models.CharField(max_length=200)
    content = models.TextField()
    def __str__(self):
        return self.title
2 forms.py
from django import forms
from .models import Post
class PostForm(forms.ModelForm):
    class Meta:
        model = Post
        fields = ['title', 'content']
3 views.py
from django.shortcuts import render
from .forms import PostForm
def add_post(request):
```



```

if request.method == "POST":
    form = PostForm(request.POST)
    if form.is_valid():
        post = form.save()
        return render(request, 'posts/result.html', {
            'post': post
        })
    else:
        form = PostForm()
    return render(request, 'posts/add_post.html', {
        'form': form
    })
4 urls.py
from django.urls import path
from . import views
urlpatterns = [
    path('add/', views.add_post, name='add_post'),
]
5 add_post.html
{% extends 'base.html' %}
{% block content %}
<h2>Добавить публикацию</h2>
<form method="POST">
    {% csrf_token %}
    <div class="mb-3">
        <label>Заголовок</label>
        <input type="text" name="title" class="form-control">
    </div>
    <div class="mb-3">
        <label>Текст публикации</label>
        <textarea name="content" class="form-control"></textarea>
    </div>
    <button type="submit" class="btn btn-primary">Сохранить</button>
</form>
{% endblock %}
6 result.html
{% extends 'base.html' %}
{% block content %}
<div class="card">
    <div class="card-body">
        <h3>Публикация добавлена</h3>
        <p><strong>Заголовок:</strong> {{ post.title }}</p>
        <p><strong>Текст:</strong> {{ post.content }}</p>
        <a href="{% url 'add_post' %}" class="btn btn-secondary">
            Добавить ещё

```

```

</a>
</div>
</div>
{% endblock %}

```

Аутентификация и авторизация пользователя

Django предоставляет систему аутентификации и авторизации («permission») пользователя, реализованную на основе фреймворка работы с сессиями, который мы рассматривали в предыдущей части. Система аутентификации и авторизации позволяет Вам проверять учётные данные пользователей и определять какие действия какой пользователь может выполнять.

Django включает в себя встроенные модели для Пользователей и Групп (основной способ применения прав доступа для более чем одного пользователя), непосредственно саму систему прав доступа (permissions)/флаги, которые определяют может ли пользователь выполнить задачу, с какой формой и отображением для авторизованных пользователей, а также получить доступ к контенту с ограниченным доступом.

Для начала создадим новый Django-проект и настроим отображение страницы (рисунок 26):

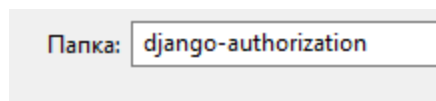


Рисунок 26 - Создание папки проекта django-authorization

После того, как мы создали папку проекта, откроем её в Visual Studio Code, установим виртуальное окружение и активируем его. Для этого пропишем следующие команды (рисунок 27):

```
python -m venv .venv
.venv/scripts/activate
```

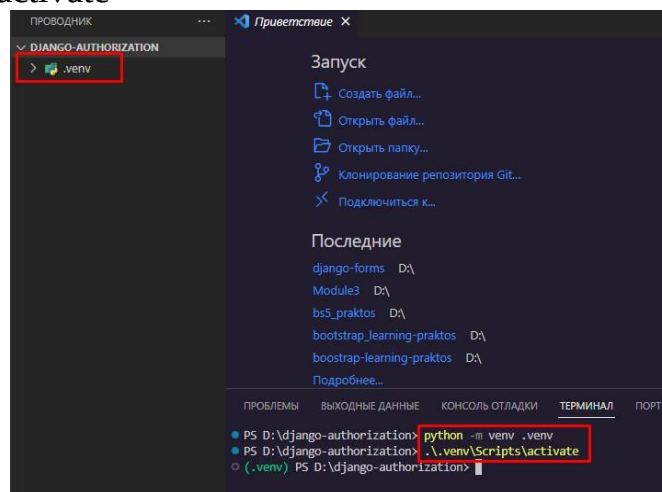
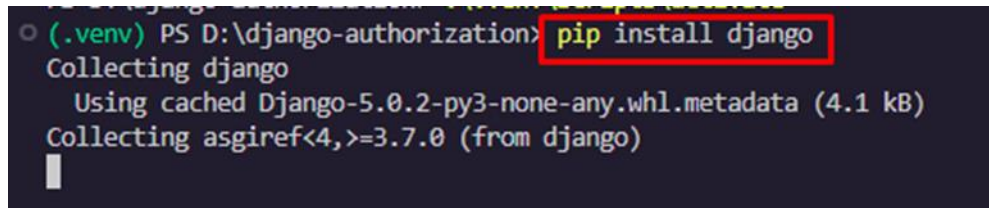


Рисунок 27 - Настройка виртуального окружения

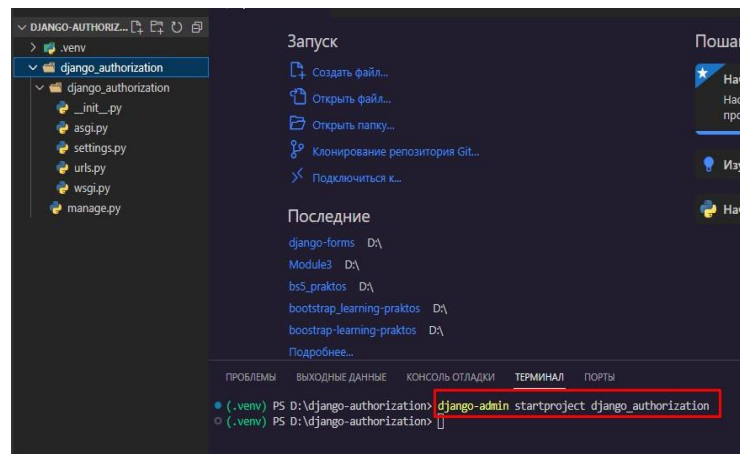
Установим Django командой «pip install django» (рисунок 28):



```
(.venv) PS D:\django-authorization> pip install django
Collecting django
  Using cached Django-5.0.2-py3-none-any.whl.metadata (4.1 kB)
Collecting asgiref<4,>=3.7.0 (from django)
```

Рисунок 28 - Установка Django

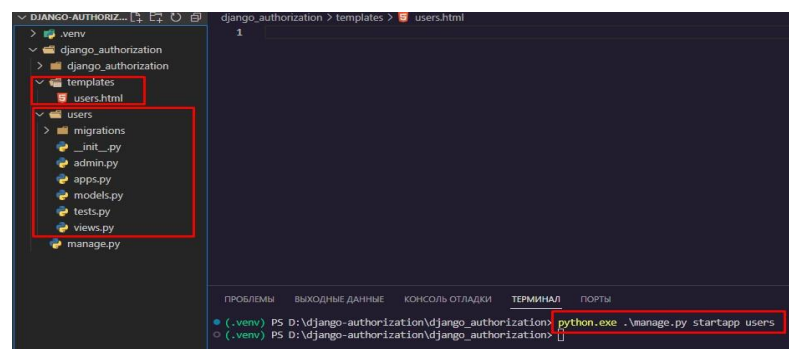
После установки Django необходимо создать новый проект. Для этого пропишем команду «django-admin startproject django_authorization» (рисунок 29):



```
(.venv) PS D:\django-authorization> django-admin startproject django_authorization
```

Рисунок 29 - Создание нового проекта django_authorization

Создадим новый django-приложение (django-app) и создать папку для шаблонов «templates» (рисунок 30):



```
(.venv) PS D:\django-authorization> python.exe manage.py startapp users
```

Рисунок 30 - Создание нового django-приложения (users) и папки templates

Откроем файл models.py приложения users и импортируем в него абстрактный класс из django.contrib.auth.models и пропишем пока-что пустой класс для будущего пользователя (рисунок 31):

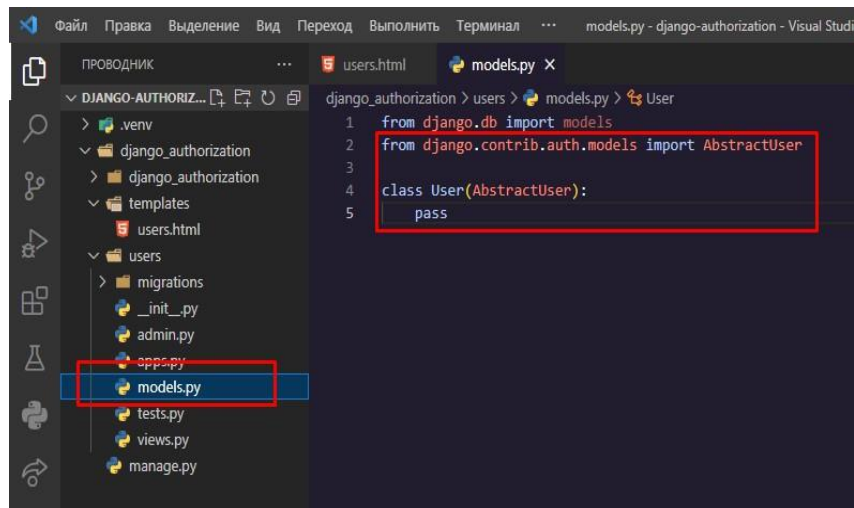


Рисунок 31 - Импорт AbstractUser из django.contrib.auth.models и создание класса User на его основе

В файле settings.py необходимо указать какая модель будет использоваться для авторизации. Пропишем следующий код в файле настроек (рисунок 32):

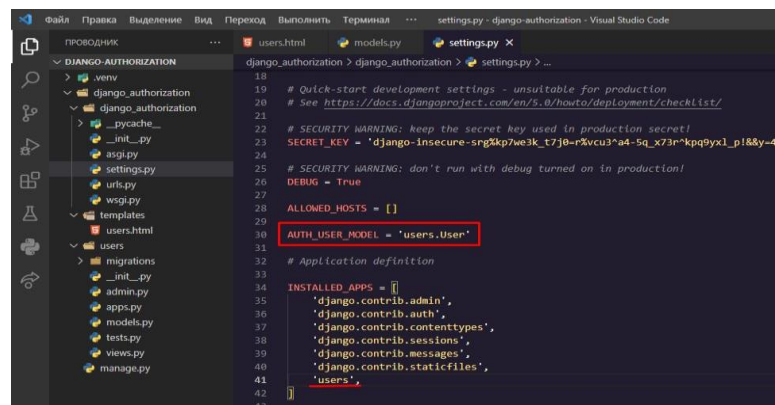


Рисунок 32 - Добавление AUTH_USER_MODEL в файле settings.py

Т.к. мы добавили новую модель, необходимо провести и выполнить миграции, чтобы изменения вступили в силу (рисунок 8):

```
(.venv) PS D:\django-authorization> cd .\django_authorization\
(.venv) PS D:\django-authorization\django_authorization> python.exe .\manage.py makemigrations
Migrations for 'users':
  users\migrations\0001_initial.py
    - Create model User
(.venv) PS D:\django-authorization\django_authorization> python.exe .\manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions, users
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0001_initial... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying users.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying sessions.0001_initial... OK
(.venv) PS D:\django-authorization\django_authorization>
```

Рисунок 33 - Проведение миграций

Создадим суперпользователя. Для этого пропишем команду «python manage.py createsuperuser», введем username, почту (при желании) и укажем пароль (помните, что при вводе пароля, он не будет отображаться в консоли) (рисунки 34-35):

```
(.venv) PS D:\django-authorization\django_authorization> python.exe .\manage.py createsuperuser
Username: 
```

Рисунок 34 - Создание суперпользователя

```
(.venv) PS D:\django-authorization\django_authorization> python.exe .\manage.py createsuperuser
Username: 
Email address: 
Password: 
Password (again): 
This password is too common.
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.
(.venv) PS D:\django-authorization\django_authorization>
```

Рисунок 35 - Суперпользователь успешно создан

Необходимо указать нашу папку с шаблонами (templates) в settings.py. Можно указать, как угодно (рисунок 36):

- можно напрямую через «templates»;
- можно создать отдельную константу для хранения директории папки templates:

```
TEMPLATE_DIR = os.path.join(BASE_DIR, 'templates')
```

- а можно напрямую, указав BASE_DIR / 'templates'

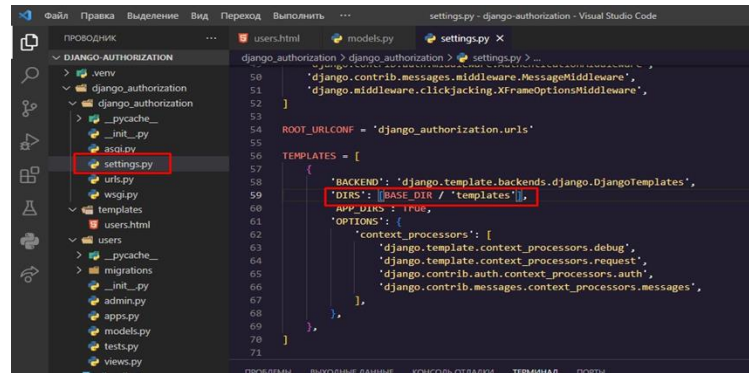


Рисунок 36 - Добавление директории для шаблонов (tempaltes)

Перейдем в base.html и сделаем так, чтобы на странице отображался текущий пользователь (рисунок 37).

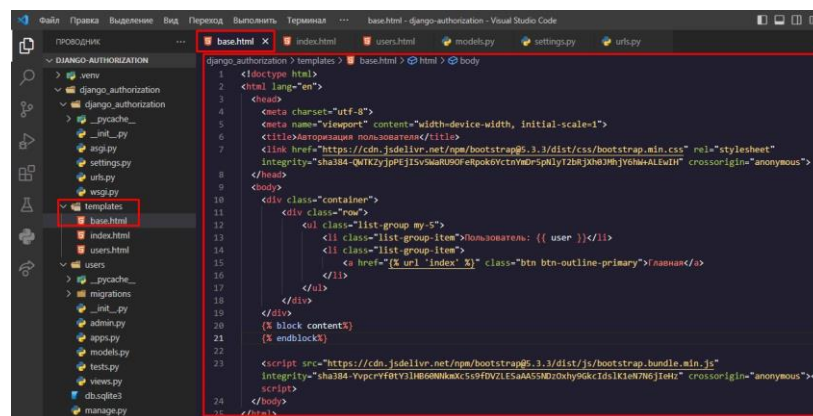


Рисунок 37 - Файл base.html

Изменим также файл index.html, будем расширяться от базового шаблона и добавим любую информацию для страницы, чтобы она не была слишком пустой (рисунок 38):

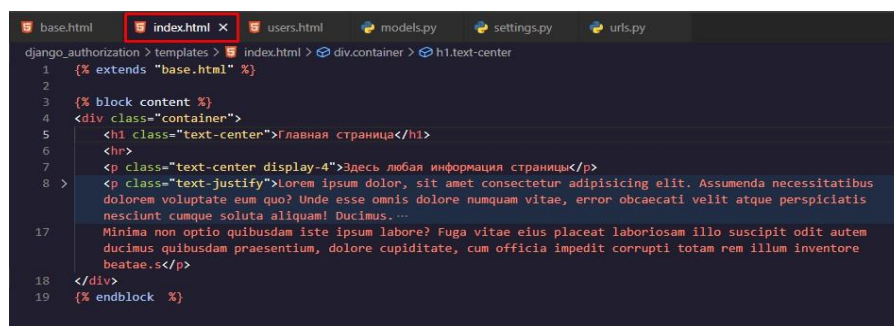


Рисунок 38 - Файл index.html

После проделанных действий, необходимо добавить маршрут для index.html. Раньше, мы делали это через отдельную вью-функцию, где просто возвращали

render страницы. Поступим более профессионально и будем использовать `TemplateView`.

TemplateView — это класс, который служит для обработки шаблонов и отправки результата пользователю (рисунок 39).

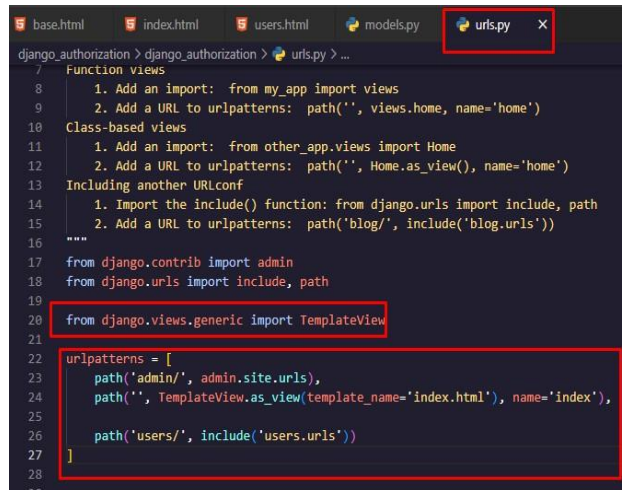


Рисунок 39 - Главный файл urls.py

Дополнительно создаем файл **urls.py** внутри приложения **users** (рисунок 40):

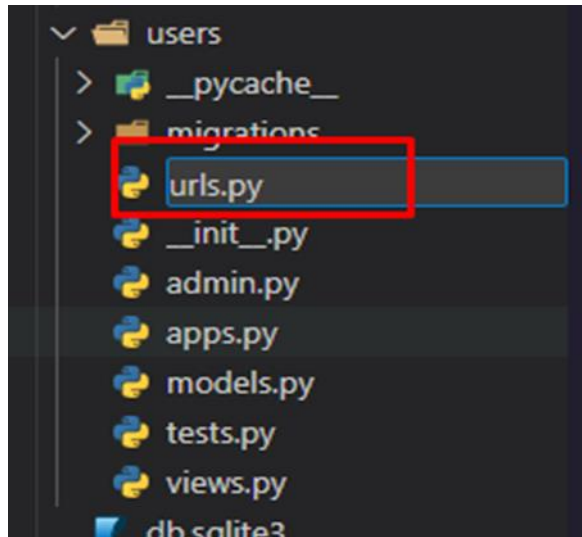


Рисунок 40 - Создание файла urls.py в приложении users

Сделаем так, чтобы мы могли просмотреть все маршруты, которые представляет Django для авторизации. Воспользуемся «include».

Функция **include()** в Django позволяет определить вложенные маршруты или подмаршруты для некоторого маршрута.

Пропишем следующий код для файла urls.py приложения users (рисунок 41):

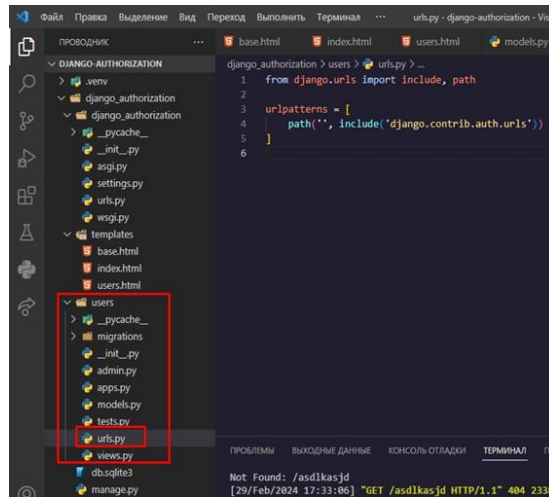


Рисунок 41 - Файл urls.py в приложении users

Если мы перейдем в адресной строке по пути users/ (который мы указали в главное urls.py файле), то можно заметить, что Django предлагает нам много различных путей (авторизоваться, выйти с аккаунта, смена пароля, подтверждение смены пароля, сброс пароля, подтверждение сброса пароля, сброс, подтверждение сброса и многое другое) (рисунок 42).

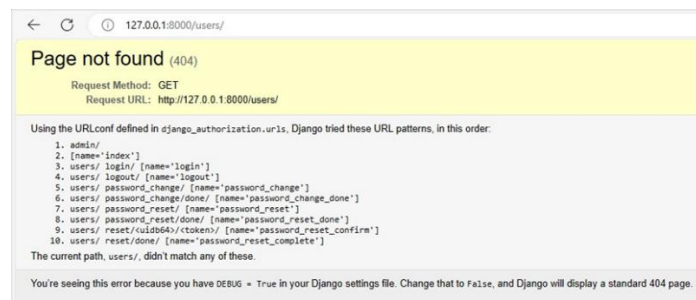


Рисунок 42 - Маршруты, предоставляемые для работы с авторизацией в Django

Далее, посмотрим, какие представления отвечают за отображение данных страниц. Для этого, можно зажать Control (CTRL) и нажать на **urls.** (где путь 'django.contrib.auth.urls') (рисунок 43):

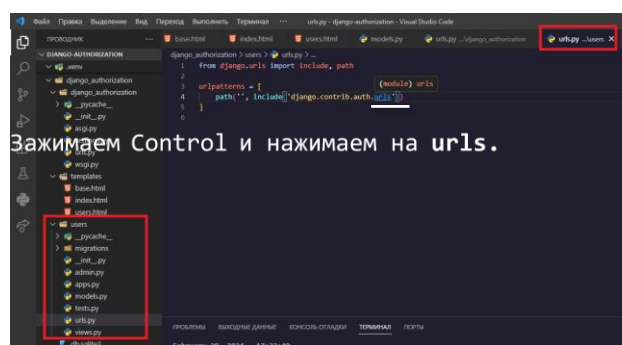


Рисунок 43 - Совет, как смотреть исходный код функций в Visual Studio Code

При переходе в данный файл, мы видим данные представления. Здесь видны урлы (urls), и у каждого url'a есть свое представление, которое обрабатывает запрос. Здесь отсутствует обработка регистрации пользователей, но мы это легко исправим, создав свою (рисунок 44).

```
from django.contrib.auth import views
from django.urls import path

urlpatterns = [
    path("login/", views.LoginView.as_view(), name="login"),
    path("logout/", views.LogoutView.as_view(), name="logout"),
    path(
        "password_change/", views.PasswordChangeView.as_view(), name="password_change"
    ),
    path(
        "password_change/done/",
        views.PasswordChangeDoneView.as_view(),
        name="password_change_done",
    ),
    path("password_reset/", views.PasswordResetView.as_view(), name="password_reset"),
    path(
        "password_reset/done/",
        views.PasswordResetDoneView.as_view(),
        name="password_reset_done",
    ),
    path(
        "reset/<uidb64>/<token>/",
        views.PasswordResetConfirmView.as_view(),
        name="password_reset_confirm",
    ),
    path(
        "reset/done/",
        views.PasswordResetCompleteView.as_view(),
        name="password_reset_complete",
    ),
]
```

Рисунок 44 - Файл urls.py, отвечающий за авторизацию, находящийся внутри Django (исходники фреймворка)

Изменим немного внешнее оформление страницы base.html, добавим стили Bootstrap (рисунок 45):

```
<div class="container">
  <div class="row">
    <ul class="list-inline my-5">
      <li class="list-inline-item">Пользователь: {{ user }}</li>
      <li class="list-inline-item">
        <a href="{% url 'index' %}" class="btn btn-outline-primary">Главная</a>
      </li>
      <li class="list-inline-item">
        <a href="{% url 'register' %}" class="btn btn-outline-primary">Регистрация</a>
      </li>
    </ul>
  </div>
</div>
```

Рисунок 45 - Изменение базового шаблона base.html

Создадим страницу для регистрации register.html в папке templates (рисунок 46):

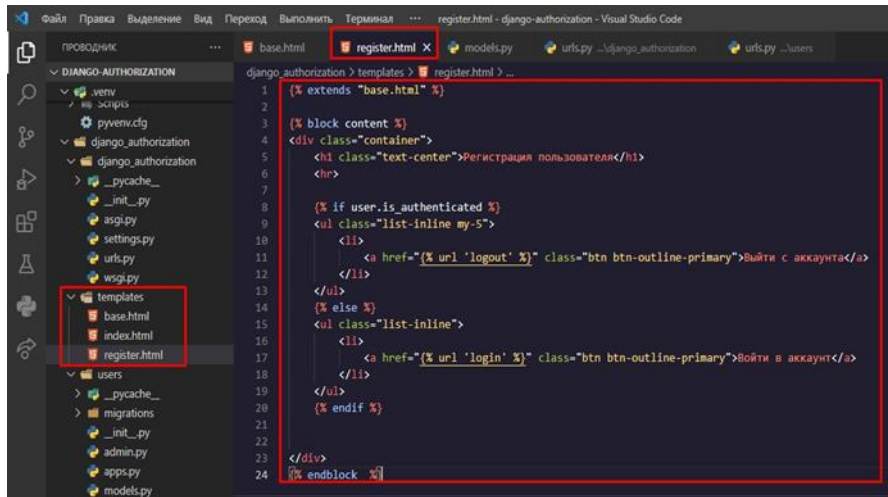


Рисунок 46 - Файл register.html

Перейдем на страницу регистрации и посмотрим, что будет если нажать на ссылку «Регистрация» (рисунок 47):

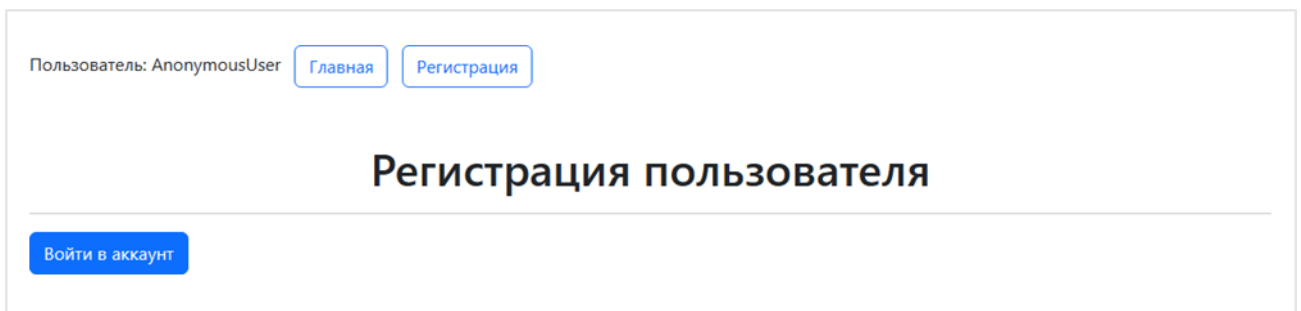


Рисунок 47 - Отображение страницы register.html

Попробуем нажать на ссылку «Войти в аккаунт» (рисунок 48):



Рисунок 48 - Ошибка TemplateDoesNotExist

Мы видим ошибку, возникающая потому, что у нас нет отображения для

представления login.html (которая, в свою очередь находится в каталоге registration). Мы должны создать все именно также! (рисунок 49):

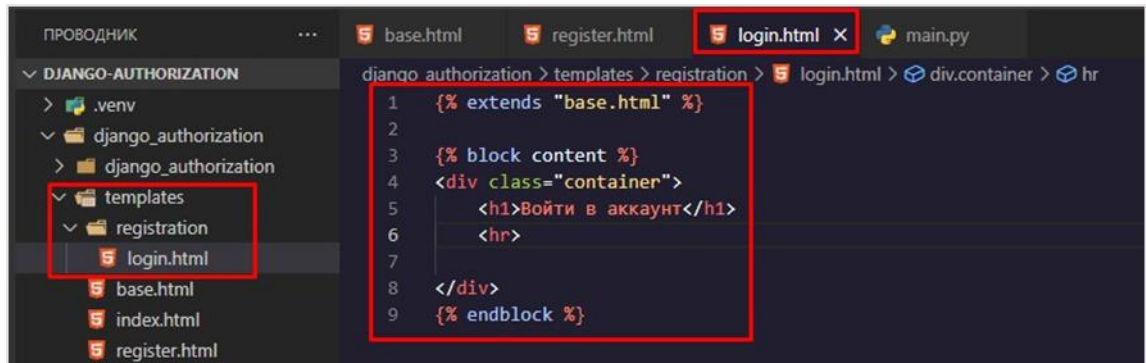


Рисунок 49 - Создание представления для отображения login.html в папке registration

Если мы посмотрим на страницу, то ошибки больше нет (рисунок 50).

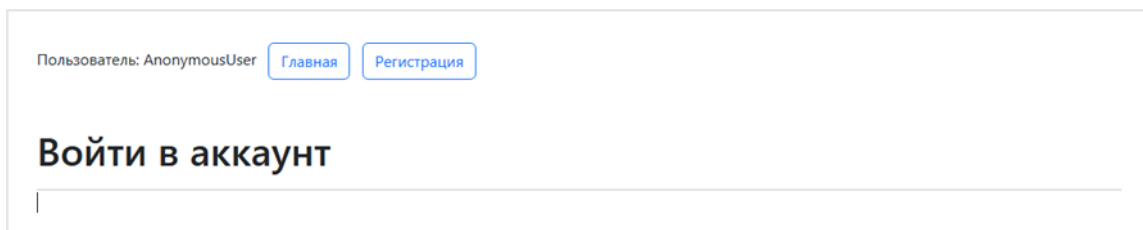


Рисунок 50 - Отображение страницы login.html

Если теперь, мы вернемся в файл urls.py, нажмем на urls (с зажатым CTRL) и перейдем в представление, которое отвечает за отображение авторизации (login), то мы увидим, что она работает за счет класса LoginView. Нажмем на него с зажатым CTRL (рисунок 51).

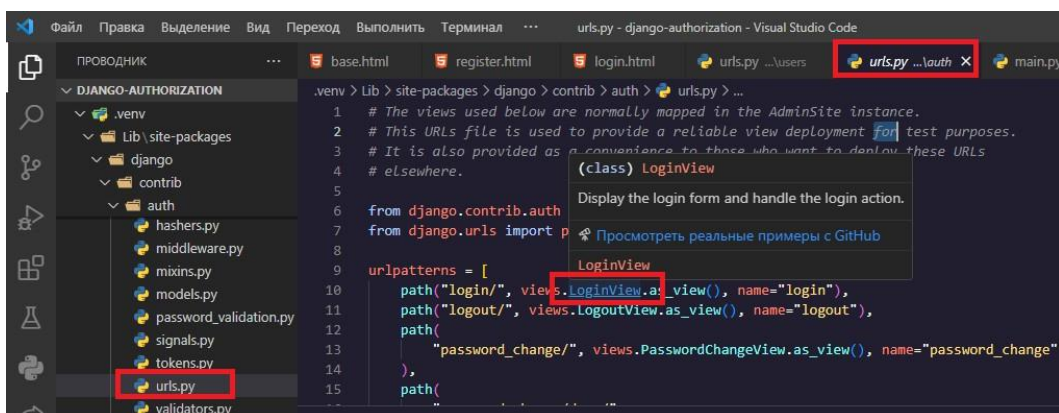


Рисунок 51 - Файл urls.py в исходниках Django

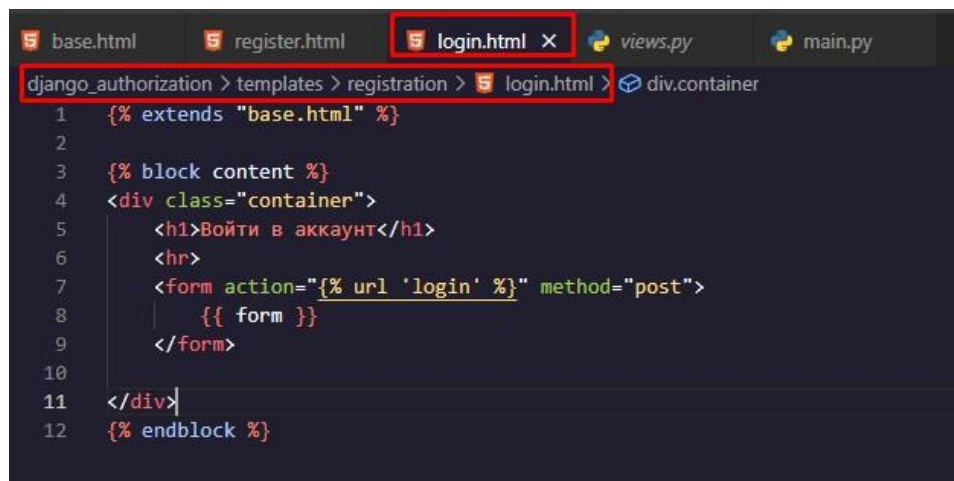
После нажатия на LoginView увидим следующий код (рисунок 52):

```
class LoginView(RedirectURLMixin, FormView):
    """
    Display the login form and handle the login action
    """

    form_class = AuthenticationForm
    authentication_form = None
    template_name = "registration/login.html"
    redirect_authenticated_user = False
    extra_context = None
```

Рисунок 52 - LoginView в исходниках Django

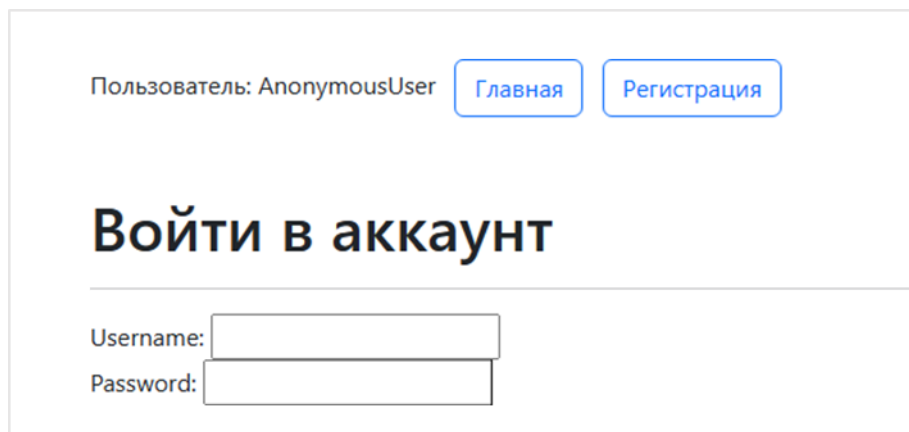
Мы увидим, что здесь используется форма AuthenticationForm. Мы также будем использовать для авторизации. Перейдем в файл login.html и добавим в него следующий код, чтобы отобразить полностью форму авторизации (рисунок 53).



```
1 {% extends "base.html" %}
2
3 {% block content %}
4 <div class="container">
5     <h1>Войти в аккаунт</h1>
6     <hr>
7     <form action="{% url 'login' %}" method="post">
8         {{ form }}
9     </form>
10
11 </div>
12 {% endblock %}
```

Рисунок 53 - Файл login.html

Перейдем на страницу авторизации и увидим результат (рисунок 54):



Пользователь: AnonymousUser [Главная](#) [Регистрация](#)

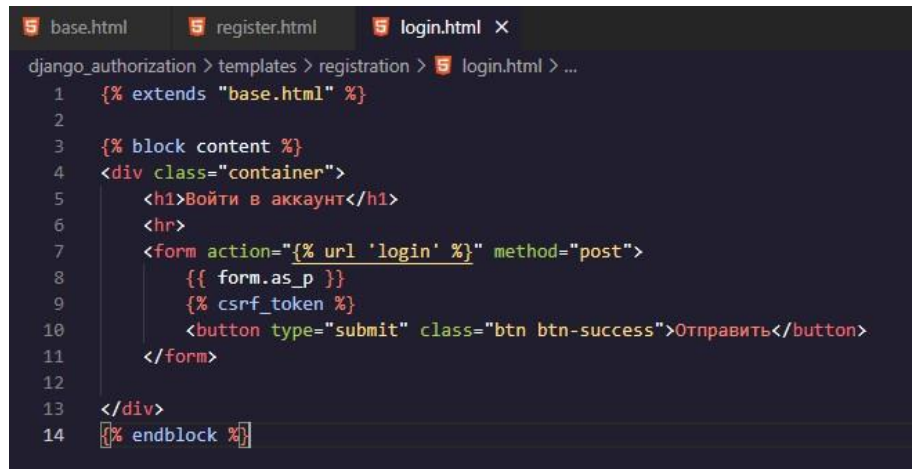
Войти в аккаунт

Username:

Password:

Рисунок 54 - Страница авторизации

Нужно добавить CSRF_TOKEN, ведь мы работаем с пост-запросами (post) и кнопку «Отправить» (рисунки 55-56).

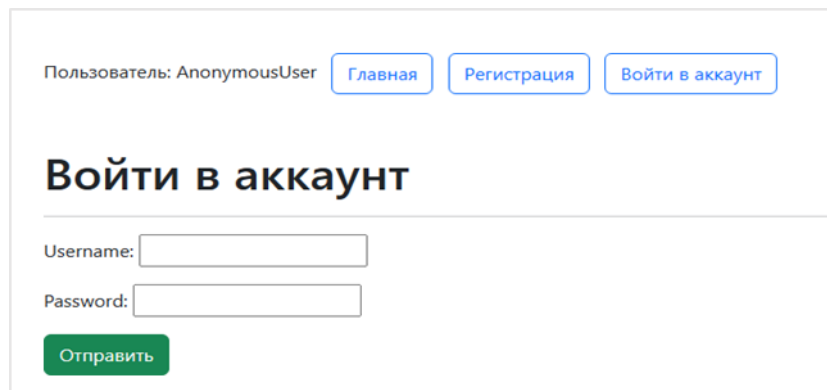


```

1  {% extends "base.html" %}
2
3  {% block content %}
4  <div class="container">
5      <h1>Войти в аккаунт</h1>
6      <hr>
7      <form action="{% url 'login' %}" method="post">
8          {{ form.as_p }}
9          {% csrf_token %}
10         <button type="submit" class="btn btn-success">Отправить</button>
11     </form>
12
13 </div>
14 {% endblock %}

```

Рисунок 55 - Добавление csrf_token'a и кнопки «Отправить»



Пользователь: AnonymousUser Главная Регистрация Войти в аккаунт

Войти в аккаунт

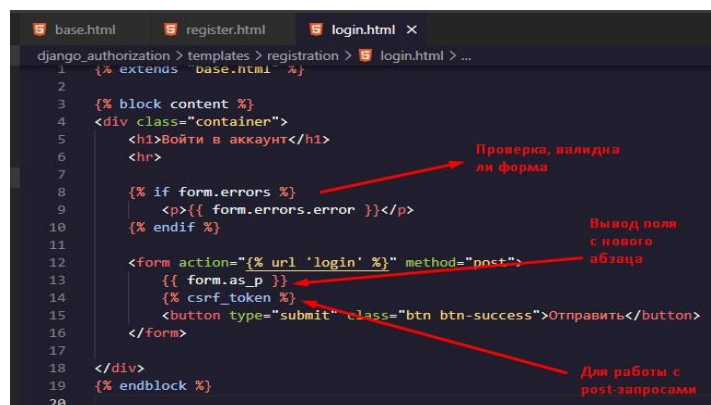
Username:

Password:

Рисунок 56 - Страница авторизации с кнопкой

Но, если мы заполним поля, и они будут некорректные, ничего не произойдет. Нужно обрабатывать ошибку, если данные будут не валидные.

Для этого, добавим условие if, и будем проверять ошибка. А также, будем выводить поля формы с нового абзаца, для этого используем {{ form.as_p }} (рисунок 57):



```

1  {% extends "base.html" %}
2
3  {% block content %}
4  <div class="container">
5      <h1>Войти в аккаунт</h1>
6      <hr>
7
8      {% if form.errors %}
9          <p>{{ form.errors.error }}</p>
10      {% endif %}
11
12      <form action="{% url 'login' %}" method="post">
13          {{ form.as_p }}
14          {% csrf_token %}
15          <button type="submit" class="btn btn-success">Отправить</button>
16      </form>
17
18 </div>
19
20 {% endblock %}

```

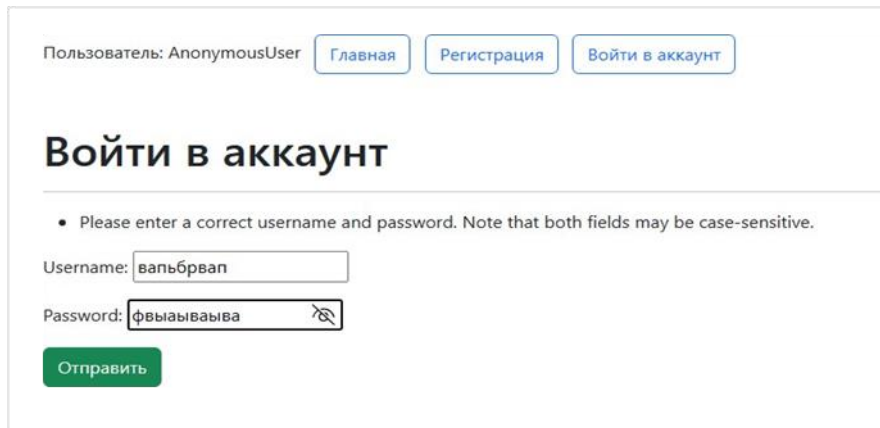
Проверка, валидна ли форма (на строку 8-10)

Вывод поля с нового абзаца (на строку 13)

Для работы с post-запросами (на строку 12)

Рисунок 57 - Редактирование кода для файла login.html

Попробуем ввести любые данные для авторизации (рисунок 58):



Пользователь: AnonymousUser [Главная](#) [Регистрация](#) [Войти в аккаунт](#)

Войти в аккаунт

- Please enter a correct username and password. Note that both fields may be case-sensitive.

Username:

Password:

[Отправить](#)

Рисунок 58 - Ввод невалидных данных для авторизации

Текст выводится на английском, это происходит из-за того, что в настройках нашего проекта (settings.py) указан языковой код 'en-us' (код языка для английского языка США), необходимо указать код для русского языка. Код русского языка – ru (Согласно – ISO 639). Исправим файл settings.py следующим образом (рисунок 59):

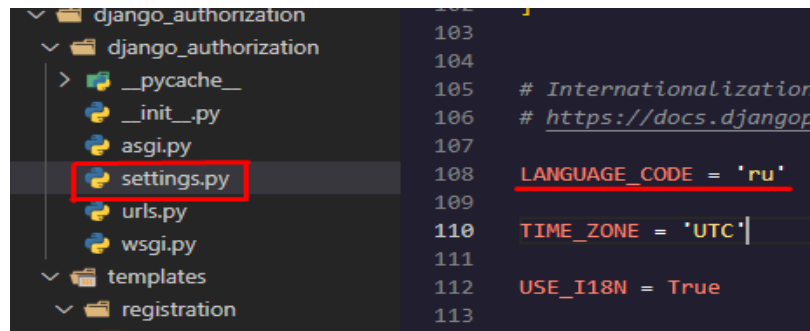
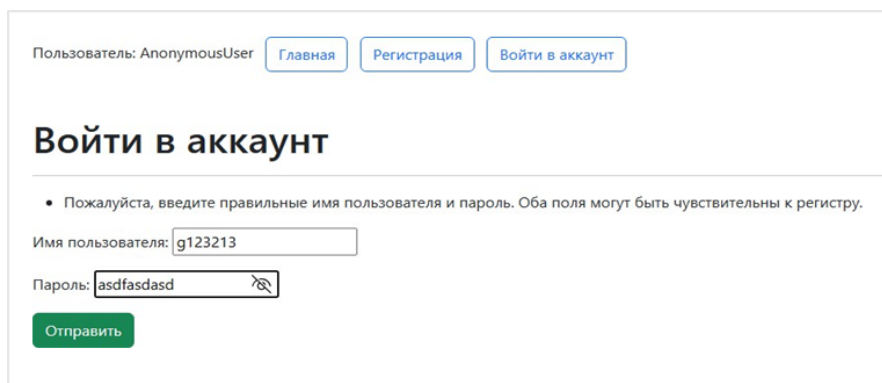


Рисунок 59 - Исправление языка в файле settings.py

После внесения изменений, язык будет изменен на русский (рисунок 60).



Пользователь: AnonymousUser [Главная](#) [Регистрация](#) [Войти в аккаунт](#)

Войти в аккаунт

- Пожалуйста, введите правильные имя пользователя и пароль. Оба поля могут быть чувствительны к регистру.

Имя пользователя:

Пароль:

[Отправить](#)

Рисунок 60 - Отображение ошибок при авторизации на русском языке

Как вы помните, мы создавали суперпользователя. Попробуем авторизоваться под суперпользователем.

Вводим данные суперпользователя и нажимаем «Отправить» (рисунок 36):

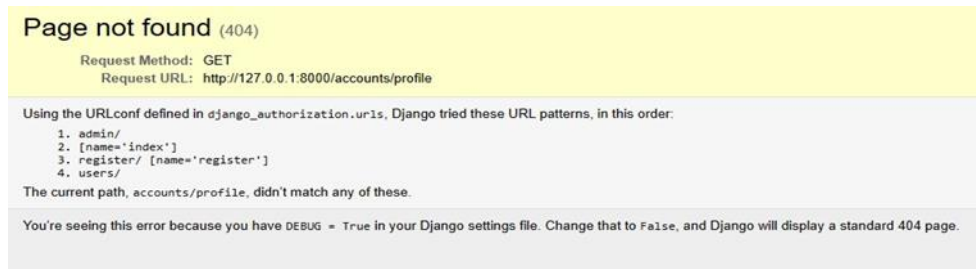


Рисунок 61 - Ошибка 404 (Page not found)

Происходит это из-за того, что нет данного пути, его можно как создать, так и ПЕРЕОПРЕДЕЛИТЬ. По-умному, нет смысла создавать новое представление просто для отправки, давайте же переопределим тот адрес, по которому нас будет направлять после авторизации. Точно также поступим и для случая, если пользователь вышел с аккаунта. Добавим LOGIN/LOGOUT REDIRECT URL'ы в файле настроек settings.py (рисунок 62):

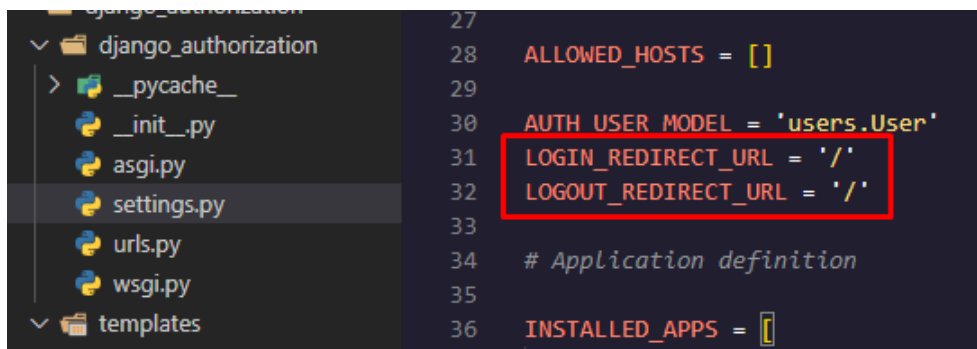


Рисунок 62- Добавление LOGIN_REDIRECT_URL и LOGOUT_REDIRECT_URL в файле settings.py

Если указать '/', переадресация будет на главную страницу.

Тот факт, что logout работает GET-запросом, был (вероятно) нарушением протокола HTTP и он позволял определенным сценариям выходить из аккаунта пользователей без их желания. Поэтому, если необходимо выходить из аккаунта пользователя, то нужно прописывать мини-шаблон формы для корректной работы выхода из аккаунта.

Изменим файл base.html и добавим прямо в него выход из аккаунта только в том случае, если мы авторизованы.

И удалим вход/выход с аккаунта со страницы регистрации (рисунок 63).

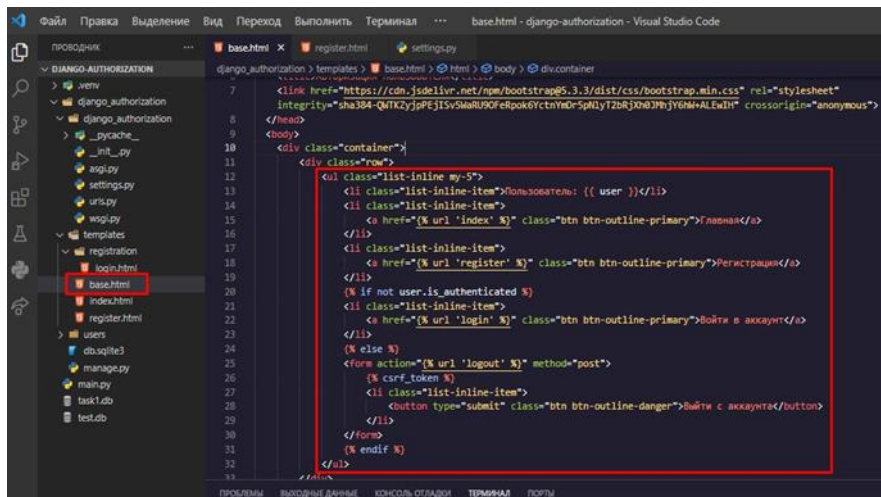


Рисунок 63 - Исправление кода для базового шаблона base.html

Изменим главный файл urls.py. Добавим маршрут для регистрации (рисунок 64):

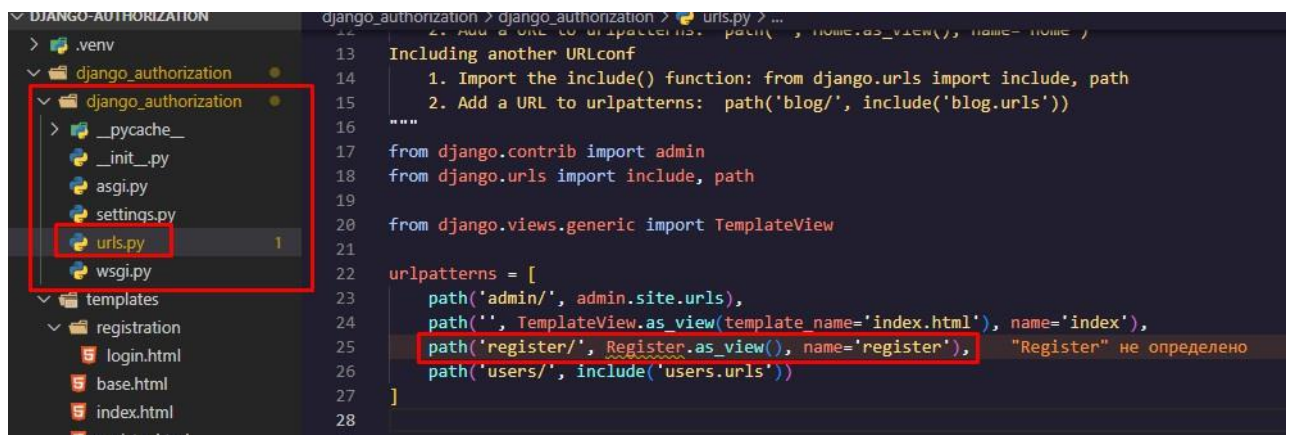


Рисунок 64 - Добавление маршрута для регистрации

Добавим выюшку для регистрации, в которой укажем переменную `template_name` и передадим ей путь до представления регистрации (`registration/register.html`). Укажем функцию **get()** для обработки запроса с методом GET, создадим в нем переменную для контекста и передадим форму для регистрации, которую импортировали из `django.contrib.auth.forms`.

Функция **get()** отвечает за обработку запроса с методом **GET**.

Аналогично, функция **post()** отвечает за обработку запроса с методом **POST** (рисунок 66).

```

django_authorization > users > views.py > Register > get
1  from django.shortcuts import render
2  from django.views import View
3  from django.contrib.auth.forms import UserCreationForm
4
5  class Register(View):
6
7      template_name = 'registration/register.html'
8
9      def get(self, request):
10         context = {
11             'form': UserCreationForm()
12         }
13         return render(request, self.template_name, context)

```

Рисунок 65 - Код файла views.py в django-приложении users

Чтобы маршрут, который мы указали в urls.py работал корректно, необходимо импортировать в данном файле класс Register, который мы только что создали (рисунок 67).

```

from users.views import Register

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', TemplateView.as_view(template_name='index.html'), name='index'),
    path('register/', Register.as_view(), name='register'),
    path('users/', include('users.urls'))
]

```

Рисунок 66 - Импорт класса для регистрации из файла views.py приложения users

Необходимо перенести файл register.html в каталог registration (рисунок 68).

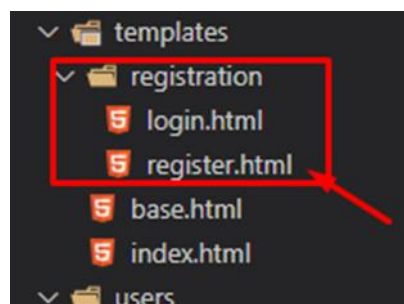


Рисунок 67 - Папка registration, содержащая в себе login.html и register.html

Перейдем на сайт, и убедимся, что страница регистрации отображается корректно (рисунок 69):

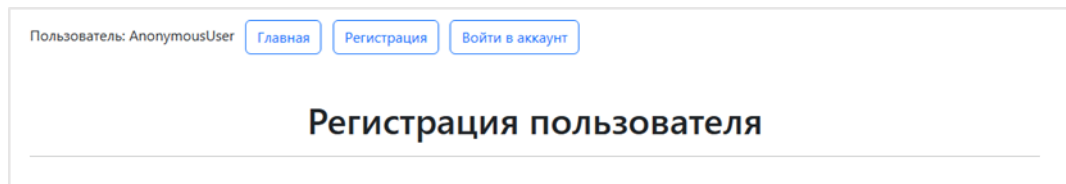


Рисунок 69 - Страница регистрации

Добавим в register.html следующий код (рисунок 70):

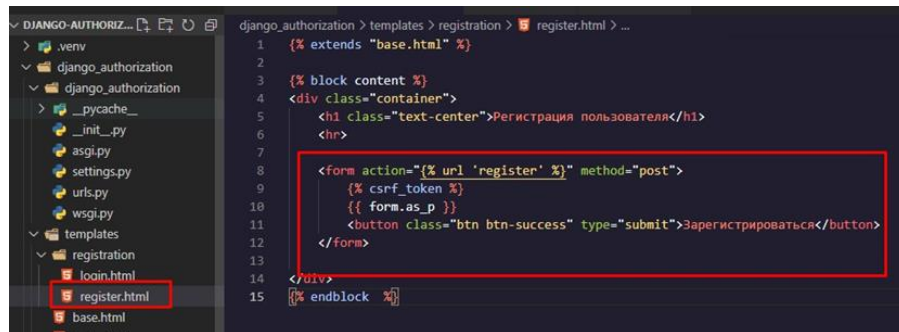


Рисунок 70 - Редактирование кода в файле регистрации

Создадим файл forms.py (рисунок 71):

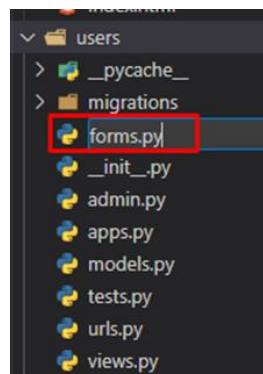


Рисунок 71 - Создание файла для форм (forms.py)

Добавим в файл форм следующий код (рисунок 72):

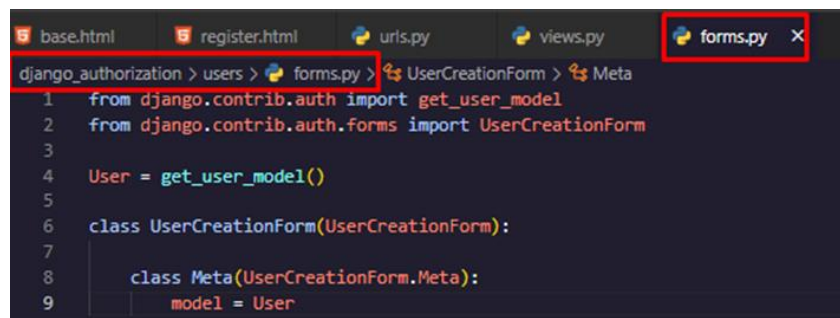


Рисунок 72 - Код файла forms.py

И добавим функцию get для регистрации, в которой будем проверять валидна ли форма, пришедшая от пользователя, если форма валидна, то берем данные из полей username и password1 и авторизовываем пользователя (рисунок 73).

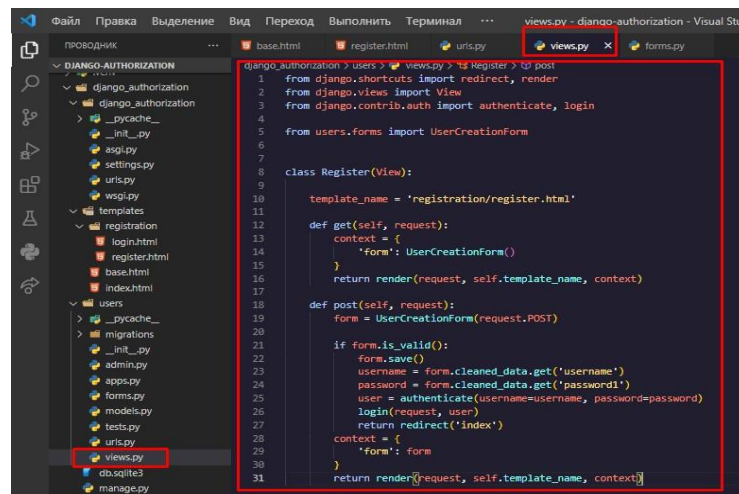


Рисунок 73 - Редактирование файла views.py (добавление функции post)

Регистрируем класс пользователя в файле admin.py, чтобы в административной панели Django мы могли просматривать пользователей (рисунок 74):

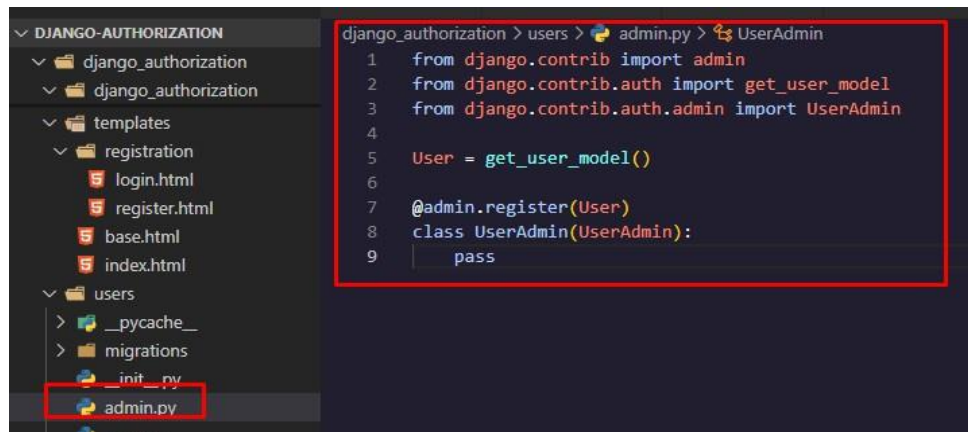


Рисунок 74 - Регистрация класса User и создание класса UserAdmin в admin.py

Зайдем после всех действий на сайт, и увидим следующую картину (рисунок 75):

Пользователь: AnonymousUser [Главная](#) [Регистрация](#) [Войти в аккаунт](#)

Регистрация пользователя

Имя пользователя: Обязательное поле. Не более 150 символов. Только буквы, цифры и символы @/./+/-/_.

Пароль:

- Пароль не должен быть слишком похож на другую вашу личную информацию.
- Ваш пароль должен содержать как минимум 8 символов.
- Пароль не должен быть слишком простым и распространенным.
- Пароль не может состоять только из цифр.

Подтверждение пароля: Для подтверждения введите, пожалуйста, пароль ещё раз.

[Зарегистрироваться](#)

Рисунок 75 - Файл регистрации с отображением всех ошибок/рекомендаций

Выглядит как-то скучно, да и все ошибки/рекомендации по заполнению полей сразу отображаются. Для более детальной настройки, изменим файл `register.html`, и применим стили Bootstrap (рисунок 76):

```

1  {% extends "base.html" %}
2
3  {% block content %}
4  <div class="container">
5    <h1 class="text-center">Регистрация пользователя</h1>
6    <div>
7      <form action="{% url 'register' %}" method="post">
8        <div class="row justify-content-center">
9          <div class="col-6">
10             <div class="form-group">
11               <label for="id_username" class="mb-2">Имя пользователя</label>
12               <input type="text" name="username" id="id_username" class="form-control mb-3" placeholder="Введите имя пользователя">
13             </div>
14             <label for="id_password1" class="mb-2">Пароль</label>
15             <input type="password" name="password1" id="id_password1" class="form-control mb-3" placeholder="Введите имя пользователя">
16             <label for="id_password2" class="mb-2">Подтверждение пароля</label>
17             <input type="password" name="password2" id="id_password2" class="form-control mb-3" placeholder="Введите имя пользователя">
18             <div class="text-center">
19               <button class="btn text-center btn-success" type="submit">Зарегистрироваться</button>
20             </div>
21           </div>
22         </div>
23       </form>
24     </div>
25   </div>
26 </div>
27 </div>
28 </div>
29 </div>
30 </div>
31 </div>
32 </div>

```

Рисунок 76 - Код файла `register.html`

Хоть и выглядит все страшно, но тут все предельно просто.

В конечном итоге, Вы получите регистрацию и авторизацию пользователя, а также отображение зарегистрированных пользователей в административной панели Django.

Реализуем довольно-таки сложную на первый взгляд логику, для отображения ошибок полей в форме. Делается это через `form.errors`.

Основная суть обработки ошибок при валидации формы: мы прописываем условие прямо внутри атрибута `class` (HTML), и здесь проверяется, есть ли ошибка в поле «Имя пользователя». `is-invalid` применяется к полю, содержащего класс `form-control` в случае возникновения какой-либо ошибки (Если ошибка есть, контур поля будет светиться красным цветом)

Далее у нас идет условие для отображения непосредственно ошибки (Внутри Django данная логика уже реализована, и нам нужно лишь правильно это написать; Django за нас будет отображать определенные ошибки) В условии мы пишем, что если в поле имени пользователя содержится ошибка, то будет выводиться первая ошибка (т.е. та, в которой пользователь ошибся; т.е. данное условие проходится по всем ошибкам в списке ошибок (см. скриншот СПИСОК ВСЕХ ОШИБОК выше))

в блоке `invalid-feedback`, которое будет отображаться ПОД вводимым полем (рисунок 77):

```
<label for="id_username" class="mb-2">Имя пользователя</label>
<input type="text" name="username" id="id_username" class="form-control mb-3 {% if form.username.errors %} is-invalid {% endif %}" placeholder="Введите имя пользователя">
{% if form.username.errors %}
  <div class="invalid-feedback">
    {{ form.username.errors.0 }}
  </div>
{% endif %}
```

Рисунок 77 - Отображение ошибок для поля username

Примеры отображения ошибок (рисунки 78-81):

Имя пользователя

gaziikk

Пользователь с таким именем уже существует.

Рисунок 78 - Ошибка «Пользователь с таким именем уже существует»

Имя пользователя

Введите имя пользователя

Обязательное поле.

Рисунок 79 - Ошибка «Обязательное поле»

Имя пользователя

текст больше 150 символовтекст больше 150 символовтекст больш

Убедитесь, что это значение содержит не более 150 символов (сейчас 975).

Рисунок 80 - Ошибка «Убедитесь, что это значение содержит не более 150 символов»

Имя пользователя

пользователь~

Введите правильное имя пользователя. Оно может содержать только буквы, цифры и знаки @./+/-/_.

Рисунок 81 - Ошибка «Введите правильное имя пользователя. Оно может содержать только буквы, цифры и знаки»

Пример решения задачи на языке программирования Python

Задача. В ранее созданном проекте `blog_site` и приложении `posts` создайте систему регистрации пользователя с использованием встроенной модели `User` Django. Реализуйте форму регистрации с полями «Имя пользователя», «Электронная почта» и «Пароль». Обеспечьте сохранение данных пользователя в базе данных и последующий переход на страницу успешной регистрации или главную страницу сайта.

Решение задачи:

```
1 forms.py (создаём форму регистрации)
from django import forms
class RegisterForm(forms.Form):
    username = forms.CharField(max_length=150)
    email = forms.EmailField()
    password = forms.CharField(widget=forms.PasswordInput)
2 views.py (логика регистрации)
from django.shortcuts import render, redirect
from django.contrib.auth.models import User
from .forms import RegisterForm
def register(request):
    if request.method == 'POST':
        form = RegisterForm(request.POST)
        if form.is_valid():
            username = form.cleaned_data['username']
            email = form.cleaned_data['email']
            password = form.cleaned_data['password']
            # создание пользователя
            user = User.objects.create_user(
                username=username,
                email=email,
                password=password
            )
            user.save()
            return redirect('register_success')
    else:
        form = RegisterForm()
        return render(request, 'posts/register.html', {'form': form})
def register_success(request):
    return render(request, 'posts/register_success.html')
3 urls.py (маршруты)
from django.urls import path
from . import views
urlpatterns = [
    path('register/', views.register, name='register'),
    path('register/success/', views.register_success, name='register_success'),
```

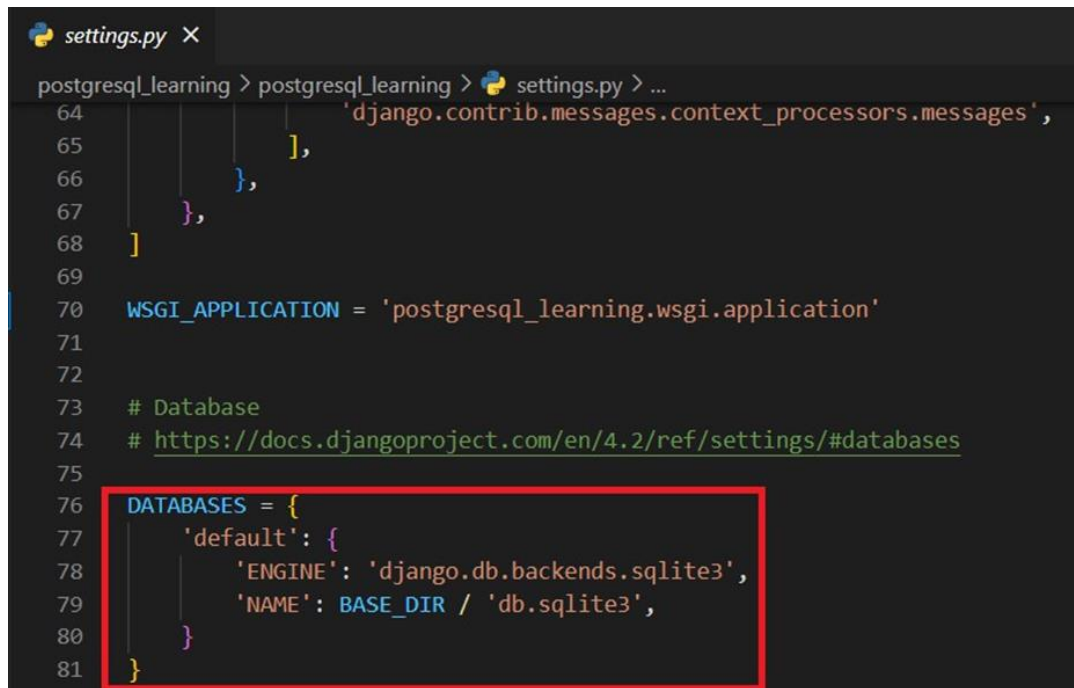
```

]
4 templates/posts/register.html
<!DOCTYPE html>
<html>
<head>
  <title>Регистрация</title>
</head>
<body>
<h2>Регистрация пользователя</h2>
<form method="post">
  {% csrf_token %}
  {{ form.as_p }}
  <button type="submit">Зарегистрироваться</button>
</form>
</body>
</html>
5 templates/posts/register_success.html
<!DOCTYPE html>
<html>
<head>
  <title>Успех</title>
</head>
<body>
<h2>Регистрация прошла успешно!</h2>
<a href="/">На главную</a>
</body>
</html>

```

Создание и настройка подключения к базе данных PostgreSQL в Django

По умолчанию Django в качестве базы данных использует SQLite. Она очень проста в использовании и не требует запущенного сервера. Все файлы базы данных могут легко переноситься с одного компьютера на другой. Однако при необходимости мы можем использовать в Django большинство распространенных СУБД (рисунок 82).



```

settings.py
postgres_learning > postgres_learning > settings.py > ...
64     'django.contrib.messages.context_processors.messages',
65 ],
66 },
67 },
68 ]
69
70 WSGI_APPLICATION = 'postgres_learning.wsgi.application'
71
72
73 # Database
74 # https://docs.djangoproject.com/en/4.2/ref/settings/#databases
75
76 DATABASES = {
77     'default': {
78         'ENGINE': 'django.db.backends.sqlite3',
79         'NAME': BASE_DIR / 'db.sqlite3',
80     }
81 }

```

Рисунок 82 - Настройка базы данных по умолчанию в файле settings.py
Поддерживаемые СУБД:

- PostgreSQL (psycopg2);
- MySQL (mysql-python);
- Oracle (cx_Oracle);
- Microsoft SQL Server (mssql-django).

Чтобы подключить PostgreSQL, его нужно для начала установить.

- переходим на сайт <https://www.postgresql.org/download/> и скачиваем PostgreSQL для вашей ОС;
- далее открываем установщик (рисунок 83).

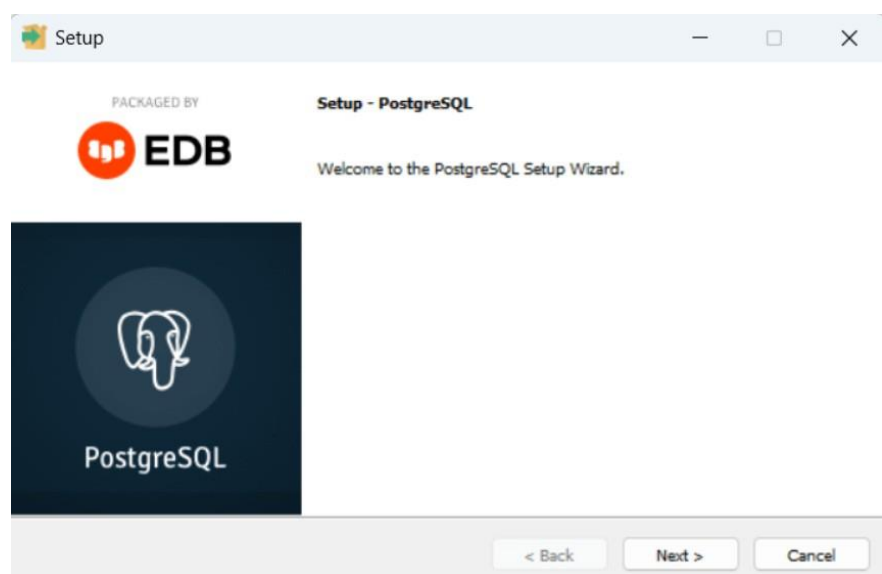


Рисунок 83 - Установка PostgreSQL

- выбираете директорию для установки СУБД;
- выбираете компоненты, идущие с СУБД (рисунок 84):

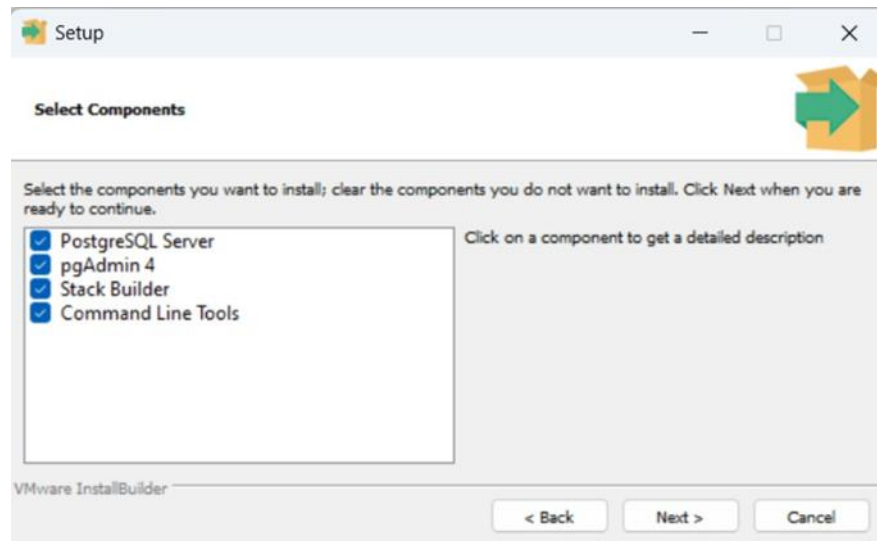


Рисунок 84 - Выбор компонентов, идущие вместе с PostgreSQL

- выбираете папку, где будут храниться базы данных;
- придумайте пароль для суперпользователя (ОБЯЗАТЕЛЬНО ЗАПОМНИТЕ ЕГО)!!! (рисунок 85);

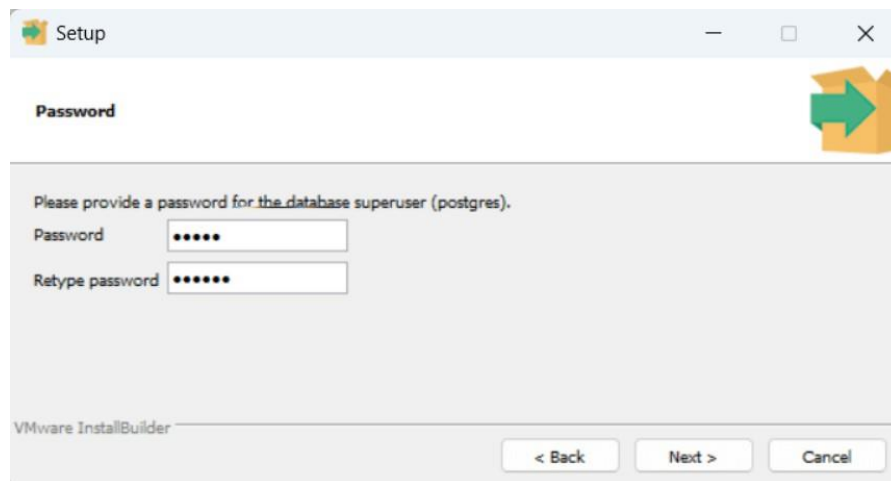


Рисунок 85 - Пароль для суперпользователя PostgreSQL

- оставляете порт по умолчанию (5432);
- в продвинутых настройках ничего не делаем и идем дальше;
- ждем, пока СУБД установится;
- поздравляю! Вы установили PostgreSQL (рисунок 86).

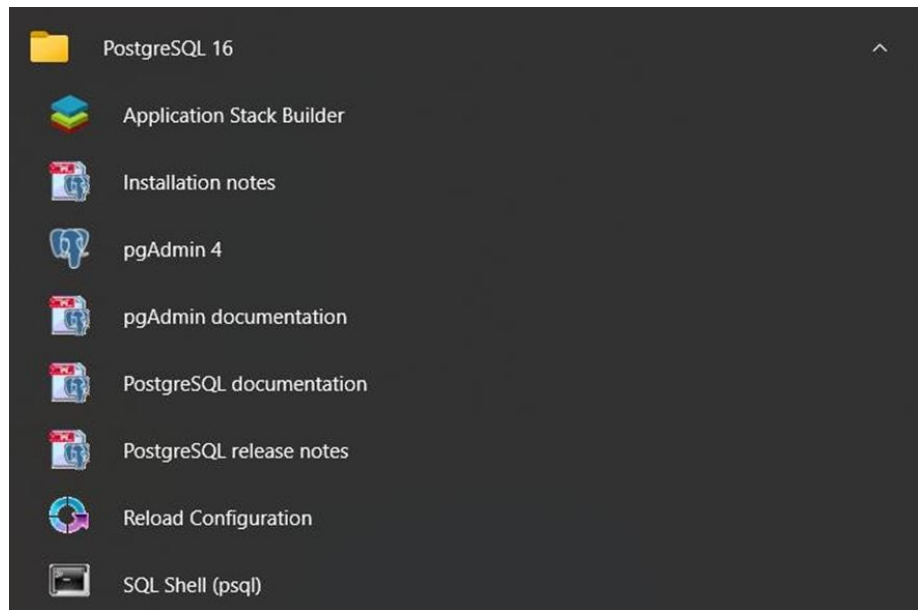


Рисунок 86 - Установленный PostgreSQL

pgAdmin – графический клиент для работы с PostgreSQL. Через него Вы можете создавать, редактировать, удалять, управлять базами данных в удобном виде. При входе в него, Вас попросят ввести пароль суперпользователя, здесь вводим тот пароль, который использовали при установке.

Для создания базы данных, сделаем действия как на скриншоте ниже (рисунок 87):

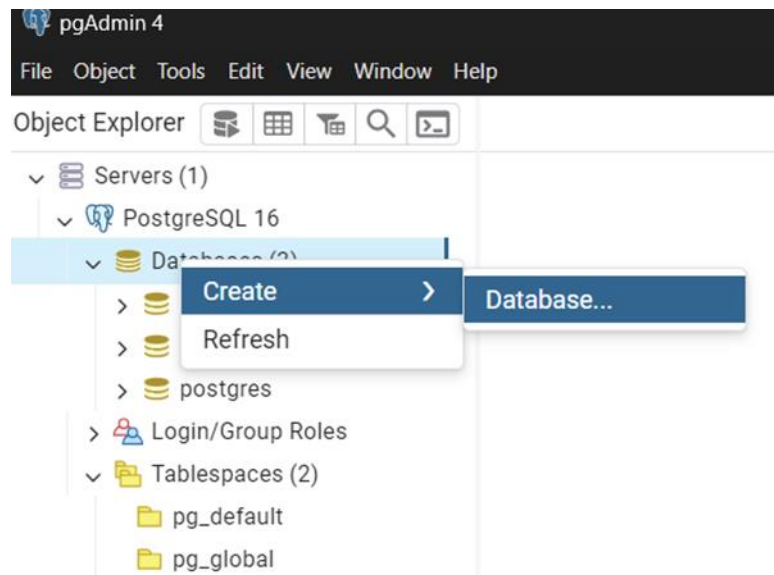


Рисунок 87 - Создание БД

После успешного создания БД нам напишет, что база данных успешно подключилась (рисунок 88).

✓ PostgreSQL 16/postgres_learning - Database connected. ✕

Рисунок 88 - Успешное соединение с БД

Для работы с запросами в PostgreSQL используется Query Tool (рисунок 89):

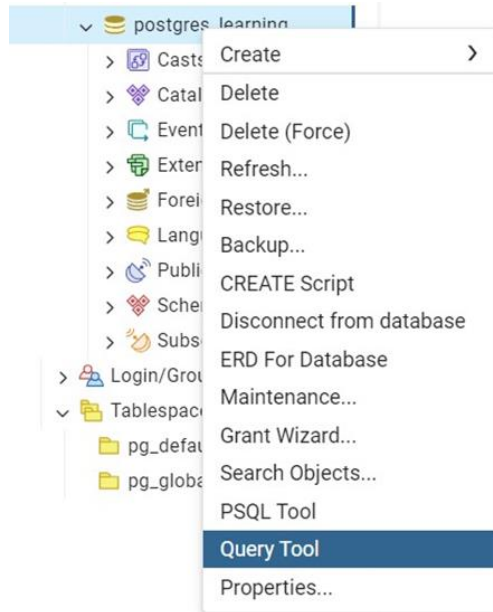


Рисунок 90 - Query tool

Вернемся к Django. Откроем файл настроек settings.py и изменим подключение с SQLite на PostgreSQL (рисунок 91).

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'postgres_learning',
        'USER': 'postgres',
        'PASSWORD': 'mypassword',
        'HOST': 'localhost',
    }
}
```

Рисунок 91 - Настройка DATABASES в settings.py для PostgreSQL

ENGINE – движок БД.

NAME – имя базы данных.

USER – имя пользователя.

PASSWORD – пароль пользователя.

HOST – хост базы данных.

Далее, установим psycopg2 для работы с PostgreSQL внутри Django (рисунок 92).

```
(.env) PS E:\Programmer\Django Projects\Pr10\postgresql_learning> pip install psycopg2
Collecting psycopg2
  Using cached psycopg2-2.9.9-cp311-cp311-win_amd64.whl.metadata (4.5 kB)
Using cached psycopg2-2.9.9-cp311-cp311-win_amd64.whl (1.2 MB)
Installing collected packages: psycopg2
Successfully installed psycopg2-2.9.9

[notice] A new release of pip is available: 24.0 -> 24.2
[notice] To update, run: python.exe -m pip install --upgrade pip
(.env) PS E:\Programmer\Django Projects\Pr10\postgresql_learning> |
```

Рисунок 92 - Установка psycopg2

После этого необходимо выполнить миграции и убедиться, что мы все сделали правильно (рисунок 93).

```
PS E:\Programmer\Django Projects\Pr10\postgresql_learning> python.exe .\manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying sessions.0001_initial... OK
```

Рисунок 93 - Выполнение миграций

Перейдем в pgAdmin и рассмотрим таблицы, которые добавились нам при развертывании проекта. Для этого, сначала нужно обновить БД (Refresh).

В остальном, вся работа она идентична, модели создаются также, миграции применяются также, все делается точно также. Но теперь, наш проект использует не SQLite, а PostgreSQL (рисунок 94-95).

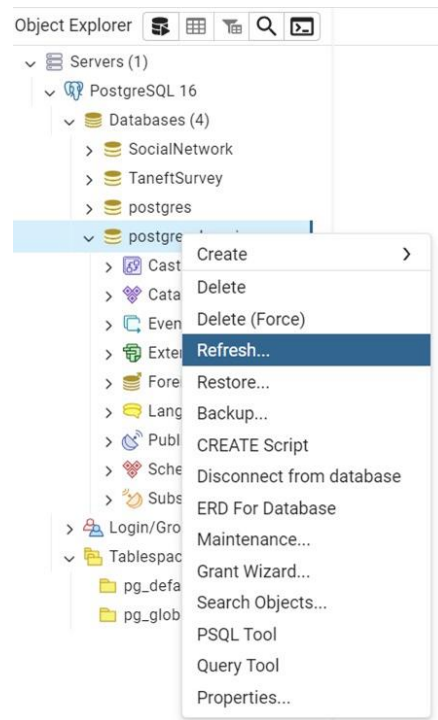


Рисунок 94 - Обновление базы данных

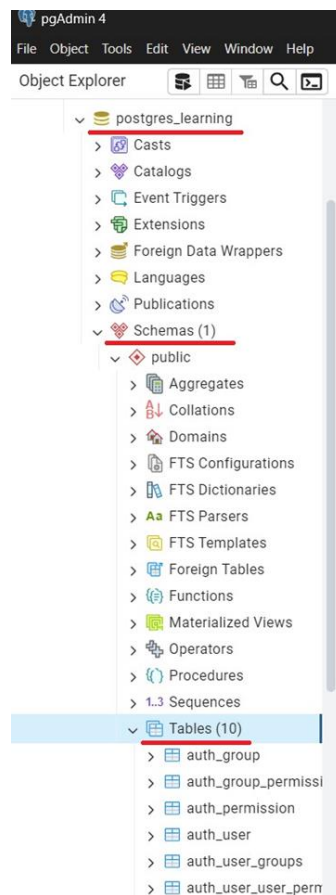


Рисунок 95 - Просмотр таблиц

Мы видим, что у нас создалось 10 новых таблиц в нашу БД (рисунок 96).

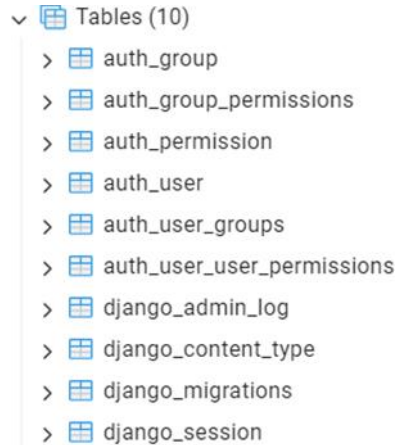


Рисунок 96 - Таблицы, добавленные в базу данных

Пример решения задачи на языке программирования Python

Задача. В ранее созданном проекте `blog_site` и приложении `posts` подключите PostgreSQL. Создайте модель `Post` с полями «Заголовок», «Текст публикации» и «Дата создания». Реализуйте сохранение записей в базе данных и их отображение на странице списка публикаций.

Решение задачи:

1 Настройка PostgreSQL (`settings.py`)

В файле `settings.py` замените `DATABASES`:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'blog_db',
        'USER': 'postgres',
        'PASSWORD': 'your_password',
        'HOST': 'localhost',
        'PORT': '5432',
    }
}
```

2 Установка драйвера PostgreSQL

```
pip install psycopg2-binary
```

3 Модель `Post` (`models.py`)

В приложении `posts/models.py`:

```
from django.db import models

class Post(models.Model):
    title = models.CharField("Заголовок", max_length=255)
    content = models.TextField("Текст публикации")
    created_at = models.DateTimeField("Дата создания", auto_now_add=True)
    def __str__(self):
        return self.title
```

4 Миграции

```
python manage.py makemigrations
```

```
python manage.py migrate
```

5 Форма добавления (forms.py)

```
from django import forms
```

```
from .models import Post
```

```
class PostForm(forms.ModelForm):
```

```
    class Meta:
```

```
        model = Post
```

```
        fields = ['title', 'content']
```

6 Представления (views.py)

```
from django.shortcuts import render, redirect
```

```
from .models import Post
```

```
from .forms import PostForm
```

```
def post_list(request):
```

```
    posts = Post.objects.all().order_by('-created_at')
```

```
    return render(request, 'posts/post_list.html', {'posts': posts})
```

```
def create_post(request):
```

```
    if request.method == 'POST':
```

```
        form = PostForm(request.POST)
```

```
        if form.is_valid():
```

```
            form.save()
```

```
            return redirect('post_list')
```

```
    else:
```

```
        form = PostForm()
```

```
    return render(request, 'posts/create_post.html', {'form': form})
```

7 URLs (posts/urls.py)

```
from django.urls import path
```

```
from . import views
```

```
urlpatterns = [
```

```
    path("", views.post_list, name='post_list'),
```

```
    path('create/', views.create_post, name='create_post'),
```

```
]
```

Подключение в главном urls.py:

```
from django.contrib import admin
```

```
from django.urls import path, include
```

```
urlpatterns = [
```

```
    path('admin/', admin.site.urls),
```

```
    path('posts/', include('posts.urls')),
```

```
]
```

8 Шаблон списка публикаций (post_list.html)

```
<h1>Список публикаций</h1>
```

```
<a href="{% url 'create_post' %}">Добавить пост</a>
```

```
{% for post in posts %}
```

```
    <div>
```

```
        <h2>{{ post.title }}</h2>
```

```

    <p>{{ post.content }}</p>
    <small>{{ post.created_at }}</small>
</div>
<hr>
{% endfor %}
9 Шаблон создания поста (create_post.html)
<h1>Создать публикацию</h1>
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Сохранить</button>
</form>

```